

Vers une formalisation adaptée à la détection d'anomalie multi-domaines

Contents

1.1	Objectifs et cadre de détection	2
1.2	État de l'art de la détection d'anomalies	3
1.2.1	Approche historique dans les réseaux traditionnels	3
1.2.2	Apprentissage profond appliqué aux réseaux 5G	4
1.2.3	Comparaison aux techniques des systèmes distribués	7
1.2.4	Synthèse	8
1.3	Méthodologie de formalisation	9
1.3.1	Extraction des features depuis la payload applicative	9
1.3.2	Transformation en graph	10
1.3.3	Encodage des arcs et nœuds	12
1.4	Détection d'anomalies dans des graphes	13
1.4.1	Introduction aux GNN	13
1.4.2	Etat de l'art des GNN	15
1.4.3	Formalisation adapté à l'utilisation de GNN dynamiques	19
1.4.4	Fonctionnement détaillé des GNN dynamique	20
1.4.5	Adaptation à notre cas d'utilisation	23
1.5	Encodage des paramètres contenus dans les arcs	25
1.5.1	Apprentissage supervisé et non supervisé	26
1.5.2	Tour d'horizon des techniques de NLP	27
1.5.3	Mise en place de l'encodage sémantique	30
1.5.4	Avantages et limites	31
1.6	Conclusion	31

Le passage aux réseaux 5G s'accompagne d'un changement de paradigme majeur, caractérisé par une architecture distribuée et une complexification des interactions entre entités. Par ailleurs, l'émergence d'anomalies sémantique, séquentielle et topologique impose de nouvelles exigences en matière d'analyse. Dans ce contexte, il devient nécessaire non seulement d'adopter une analyse fine

au niveau des paquets, mais également de prendre en compte le contexte dans lequel s'inscrivent les échanges. Or, cette problématique reste relativement récente dans le domaine des télécommunications, et les méthodes actuelles de détection d'anomalies apparaissent inadaptées à ce nouveau paradigme.

Dans ce contexte, nous proposons de représenter les échanges au sein des réseaux 5G sous la forme d'un graphe dynamique, permettant de capturer les interactions entre les différentes entités du réseau ainsi que l'évolution de ces interactions dans le temps. Cette formalisation nous permet d'exploiter des techniques issues du domaine des GNN, intrinsèquement adaptées à l'analyse des interactions complexes. Nous nous intéressons plus particulièrement aux GNN dynamiques, qui intègrent nativement la dimension temporelle.

1.1 Objectifs et cadre de détection

Dans nos travaux, nous nous plaçons du point de vue de l'opérateur d'un réseau 5G et nous nous intéressons à la détection d'anomalies en nous concentrant exclusivement sur le trafic de contrôle échangé entre les entités du réseau, qu'il s'agisse du cœur de réseau ou du RAN. Le trafic utilisateur n'est pas pris en compte, car il ne présente pas de spécificités propres à l'architecture 5G et ne relève pas de la responsabilité directe du fournisseur d'accès réseau.

En se plaçant du point de vue de l'opérateur, nous supposons que celui-ci possède l'ensemble des NF et dispose d'un accès complet au trafic de contrôle. Dans l'architecture 5G, la fonction Service Communication Proxy (SCP) agit comme un intermédiaire dans les échanges entre les différentes NF, notamment pour le routage des requêtes et la vérification des autorisations d'accès. Dans ce contexte, il est raisonnable de considérer que l'opérateur peut observer l'intégralité du trafic de contrôle en déployant des sondes au niveau des SCP, qui constituent des points d'observation privilégiés des interactions entre NF.

Ce trafic est généralement chiffré, par exemple via TLS sur le SBI ou via IPsec pour certains canaux, tels que N3 ou N4, reposant sur des protocoles spécifiques à la 5G. Cependant, en adoptant le point de vue de l'opérateur, nous supposons disposer des clés de chiffrement nécessaires pour accéder au contenu de ces échanges en clair.

L'objectif serait de pouvoir exécuter nos modèles de détection en temps réel. Malgré les débits élevés offerts par la 5G, le volume de données est majoritairement porté par le trafic utilisateur, qui n'entre pas dans le périmètre de cette étude. Le trafic de contrôle, lui, est principalement composé de messages liés aux changements d'état des entités du réseau, qu'il s'agisse d'UE ou de NF, et ont une fréquence d'occurrence plus modérée. Par conséquent, bien que le surcoût induit par nos modèles puisse avoir un impact sur les performances, il est raisonnable de supposer que les ressources disponibles sont suffisantes pour permettre un traitement en temps réel.

La remédiation des anomalies détectées ne relève pas du périmètre de cette thèse

et est considérée comme une brique fonctionnelle distincte. Nous nous concentrons exclusivement sur la détection des anomalies et générons des alertes qui pourront être exploitées de manière similaire à celles produites par un IDS classique.

L’objectif de ces travaux n’est pas de concevoir un système de détection absolu et entièrement autonome. En effet, une anomalie ne constitue pas nécessairement le signe d’une activité malveillante et peut résulter d’une mise à jour, d’une erreur de configuration ou encore d’une interconnexion avec un système tiers, comme dans le cas du roaming. Dans ce contexte, la distinction entre une anomalie bénigne et une réelle attaque requiert in fine une expertise humaine. Par conséquent, l’un des aspects essentiels de nos travaux est de proposer un système interprétable, venant assister une expertise humaine plutôt que de la remplacer.

1.2 État de l’art de la détection d’anomalies

Cette section propose un panorama des travaux de recherche consacrés à la détection d’anomalies. Dans un premier temps, nous présenterons les limites des approches traditionnelles appliquées à notre problématique, en montrant que les objectifs visés ainsi que les contraintes opérationnelles diffèrent sensiblement de notre contexte d’étude. Nous nous intéresserons ensuite aux méthodes développées pour la détection d’anomalies dans les réseaux 5G, afin d’évaluer l’état actuel de la recherche sur notre problématique spécifique et identifier les limites des travaux existants. Enfin, nous adopterons une perspective plus large en examinant les travaux issus du domaine des systèmes distribués qui, comme nous l’avons montré précédemment, présentent de nombreuses similarités structurelles et fonctionnelles avec les architectures 5G.

1.2.1 Approche historique dans les réseaux traditionnels

La détection d’attaques au sein des réseaux est historiquement assurée par des IDS. Ces systèmes reposent généralement sur une analyse du trafic agrégé sous forme de flux, plutôt qu’au niveau des paquets individuels. Cette approche permet d’extraire des métriques globales et d’effectuer des analyses de volumétrie, par exemple afin de détecter des variations significatives du volume de trafic ou repérer des signatures d’attaques connues.

Parmi les IDS les plus connus on retrouve Snort [Snort 2026], Zeek [Zeek 2026] et Suricata [Suricata 2026], qui reposent sur des règles prédéfinies pour identifier des motifs (aussi appelés signatures) d’attaques spécifiques dans le flux réseau. Ce sont la plupart du temps des outils open-source qui bénéficient d’une large communauté dont la contribution est essentielle au maintien de leur base de signatures.

Des approches fondées sur l’apprentissage automatique supervisé ont également été proposées pour la détection d’intrusion, notamment à l’aide de réseaux bayésiens [Koc 2012], de Machines à Vecteurs de Support (SVM) [Vijayanand 2018] ou encore d’arbres de décision [Moon 2017]. Ces méthodes présentent l’avantage d’être

relativement transparentes, interprétables et efficaces pour détecter des attaques déjà connues.

Parmi les approches non-supervisées, Münz et al. [Münz 2007] proposent, par exemple, de découvrir des motifs anormaux en adaptant l'algorithme K-Means au trafic réseau, permettant ainsi de regrouper les données en fonction de leur similarité sans nécessiter d'étiquettes. Néanmoins, les caractéristiques utilisées en entrée restent relativement limitées et se restreignent notamment au nombre de paquets, au volume d'octets ainsi qu'aux adresses source et destination. Une telle approche permet d'identifier des motifs anormaux liés à la volumétrie. Toutefois, elle ne prend en compte ni le contenu sémantique des paquets ni le contexte des échanges.

Ariu et al. [Ariu 2011] proposent quant à eux d'utiliser des modèles de Markov cachés afin de modéliser le trafic réseau sous forme de séquences d'octets. Ces modèles apprennent des probabilités de transition à partir de trafic bénin, puis évaluent dans quelle mesure un trafic observé s'en écarte. Cependant, les séquences considérées sont limitées à des fenêtres de tailles relativement réduites, ce qui empêche la capture de dépendances temporelles à long terme. De plus, cette approche ne permet pas de modéliser explicitement les interactions entre paquets.

Cette étude met en évidence le décalage entre les approches historiques de détection d'intrusion et les besoins spécifiques liés aux réseaux 5G. Les méthodes traditionnelles, principalement conçues pour l'analyse agrégée de flux, se révèlent peu adaptées à un environnement distribué et softwarisé. Ainsi, les IDS traditionnels restent généralement limités, soit par le manque d'expressivité des caractéristiques considérées, soit par une modélisation insuffisante des dépendances temporelles et des interactions complexes entre paquets. Ces limites justifient ainsi la nécessité de développer des méthodes plus adaptées aux spécificités des architectures 5G.

1.2.2 Apprentissage profond appliqué aux réseaux 5G

Après avoir présenté les limites des approches traditionnelles de détection d'intrusion, nous nous intéressons désormais aux méthodes spécifiquement conçues pour les réseaux 5G. L'objectif est d'analyser dans quelle mesure ces travaux prennent en compte les dimensions sémantique, séquentielle et topologique. Cette analyse permettra d'identifier les avancées récentes du domaine, mais également les limitations persistantes des approches actuelles.

Parmi les travaux de détection spécifiques à la 5G, Kale et al. [Kale 2023] proposent d'utiliser des Réseaux de Neurones Convolutionnels (CNN) en considérant les octets d'un paquet comme une image sur laquelle sont ensuite étudiées des variations structurelles. Les modèles CNN obtiennent de très bonnes performances dans de nombreux domaines, y compris dans des tâches qui ne relèvent pas intrinsèquement de la vision par ordinateur [Ganesh 2017]. Toutefois, comme ils n'interprètent pas le contenu des données, leur capacité de détection se limite à un sous-ensemble d'attaques sémantiques dans lesquelles les valeurs s'écartent

fortement du trafic normal. Ils ne permettent donc pas de détecter des variations légères de valeurs d'identifiants comme les adresses IP ou les Identifiants Utilisateurs (IMSI). De plus, le travail présenté par Kale et al. se concentre uniquement sur la détection sémantique et ne prend pas en compte le contexte des échanges. Lam et al. [Lam 2020] ajoutent un contexte temporel à leur CNN en alimentant ses sorties dans un Réseau de Neurones Récurrent (RNN). Cependant, en plus de ne pas être satisfaisant sur le domaine sémantique, leur modèle ne prend pas en compte l'identité des acteurs impliqués dans les échanges et n'est donc pas capable de détecter des attaques relationnelles.

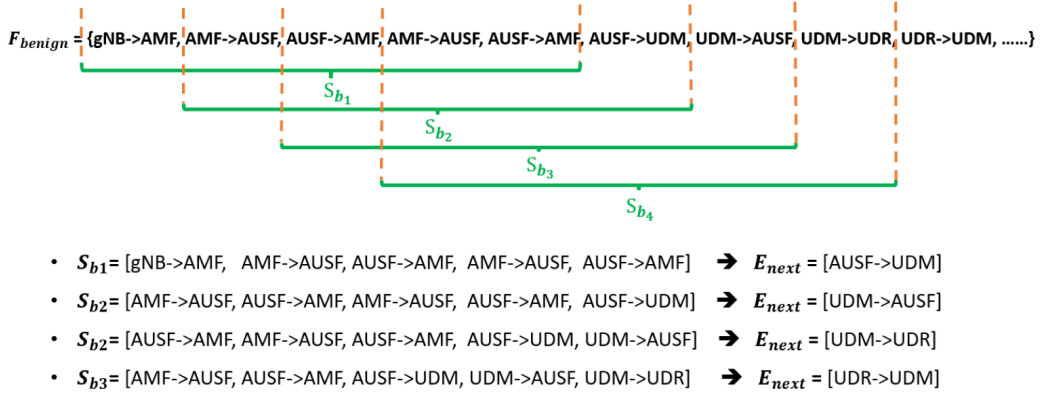


Figure 1.1: Représentation d'une séquence d'événement bénins entre différentes entités tel que présenté par ADSeq-5GCN [Tian 2023].

AutoGuard [Madi 2021] et ADSeq-5GCN [Tian 2023] modélisent les échanges réseau sous forme de séquences d'interactions entre entités du réseau tel que représenté sur figure 1.1. Ces modèles utilisent ensuite des RNN afin de capturer les dépendances temporelles et prédire les interactions futures. Leurs approches intègrent l'identité des entités communicantes, ce qui leur permet de prendre en compte, dans une certaine mesure, le contexte topologique des interactions. Cependant, elles rencontrent des difficultés à modéliser efficacement les dépendances relationnelles lorsque les interactions sont éloignées dans le temps. De plus, le principal problème est que leur formalisme n'intègre pas le contenu des paquets et ne considère donc aucunement la dimension sémantique. 5GCCGuard [Tan 2025] tente de pallier cette limitation en enrichissant leurs séquences par l'intégration de l'URL associée à chaque message. Bien qu'elle apporte une information sémantique supplémentaire, cette approche demeure très limitée car l'information propre à l'URL ne représente qu'une fraction du contenu réel des paquets et n'est applicable qu'au trafic HTTP.

Prov5GCC [Pacherkar 2024] propose de son côté de représenter le trafic sous forme de graphe de provenance. Dans cette représentation, illustrée sur la figure 1.2, les nœuds correspondent aux entités du réseau et chaque message échangé entre ces entités est modélisé par une arête, à laquelle est associé le contenu du paquet.

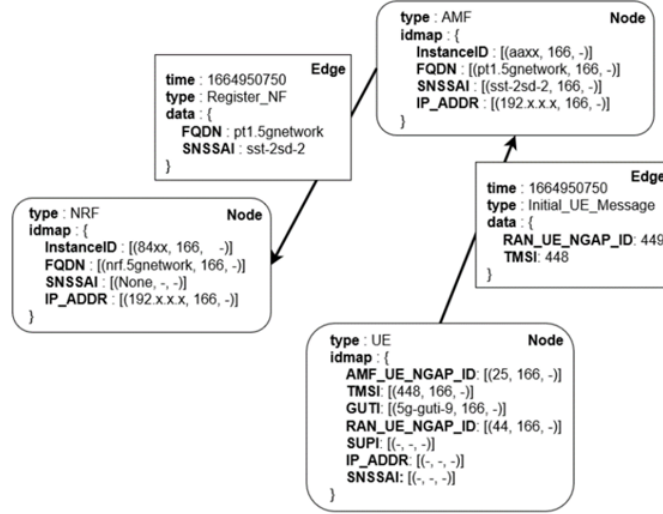


Figure 1.2: Trafic de contrôle 5G formalisé par Prov5GC [Pacherkar 2024] en tant que graphe de provenance.

La formalisation en graphe est particulièrement adaptée à l'analyse topologique des interactions du réseau. Cependant, elle ne modélise pas explicitement la dimension temporelle des échanges, ce qui restreint sa capacité à capturer l'évolution dynamique des interactions. De plus, le fait d'agrégier le payload au niveau des arêtes limite fortement l'exploitation de l'information contenue dans les paquets. Dans leur cas la payload est utilisée par un algorithme de détection heuristique spécifique à chaque attaque, ce qui rend l'approche relativement lourde et peu adaptée à des environnements complexes.

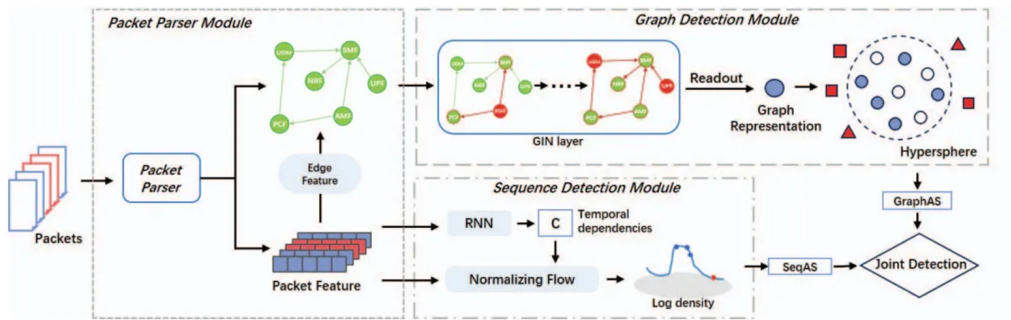


Figure 1.3: Pipeline de GSAD [Wang 2023] représentant leur formalisation ainsi que les trois modules de détections traitant respectivement les dimensions sémantique, séquentielle et topologique.

GSAD [Wang 2023] propose également de modéliser les données sous forme de graphe de manière similaire à Prov5GC [Pacherkar 2024]. La principale différence réside dans la représentation du contenu des arêtes. Au lieu de l'encoder sous forme

de dictionnaire monolithique, les auteurs le transforment en flux d’octets structuré comme une image. Cette représentation leur permet d’utiliser un CNN pour encoder chaque paquet et analyser sa dimension sémantique. Les représentations sémantiques sont ensuite injectées dans un GNN afin de modéliser les interactions entre les paquets, puis dans un RNN pour capturer les dépendances temporelles. À notre connaissance, GSAD [Wang 2023] est la seule approche de détection d’anomalies 5G qui considère simultanément les trois dimensions de détection. La principale limite de leur approche réside dans le fait que, comme évoqué précédemment, les CNN ne permettent qu’une analyse relativement superficielle de la dimension sémantique des paquets. La Figure 1.3 illustre l’architecture générale du pipeline proposé par GSAD et met en évidence les trois composantes du modèle correspondant respectivement aux trois dimensions.

Référence	Sémantique	Séquentiel	Topologique
Kale et al. [Kale 2023]	Partiel	Non	Non
Lam et al. [Lam 2020]	Partiel	Oui	Non
AutoGuard [Madi 2021]	Non	Oui	Partiel
ADSeq-5GCN [Tian 2023]	Non	Oui	Partiel
5GCGuard [Tan 2025]	Non	Oui	Partiel
Prov5GC [Pacherkar 2024]	Partiel	Non	Partiel
GSAD [Wang 2023]	Partiel	Partiel	Oui

Table 1.1: Comparaison des approches de l’état de l’art de détection d’anomalies 5G selon les dimensions considérées.

Le tableau 1.1 résume les différentes approches de détection d’anomalies dans les réseaux 5G, en mettant en évidence les dimensions sémantique, temporelle et topologique prises en compte par chacune d’elles. L’état de l’art montre que peu d’approches considèrent simultanément les trois dimensions. À travers les travaux tels que GSAD [Wang 2023] et Prov5GC [Pacherkar 2024], les GNN semblent particulièrement adaptés pour modéliser les interactions entre entités. À l’inverse, aucune des modélisations proposées ne semble réellement adaptée au traitement du contenu sémantique des paquets, celui-ci étant le plus souvent considéré comme un bloc monolithique.

1.2.3 Comparaison aux techniques des systèmes distribués

Enfin, nous prenons un peu de recul en nous intéressant aux travaux issus du domaine des systèmes distribués. Comme évoqué précédemment, les architectures 5G changent de paradigme et présentent maintenant de nombreuses similarités avec ces systèmes fortement distribués et dynamiques. Nous examinons donc dans cette section les principales approches de détection d’anomalies proposées dans ce domaine, afin d’identifier dans quelle mesure elles exploitent les spécificités de ces architectures et les enseignements qu’il est possible d’en tirer pour notre problématique.

Parmi les approches exploitant la nature distribuée des infrastructures, on retrouve notamment des travaux basés sur le Federated Learning (FL). DioT [Nguyen 2019] et VFL [Wei 2024], par exemple, promeuvent les avantages de la fédération dans les systèmes distribués, notamment en termes de passage à l'échelle, de confidentialité et de décentralisation de l'apprentissage. Cependant, leur contribution se concentre principalement sur le mécanisme de fédération lui-même. Les modèles utilisés restent relativement classiques et ne couvrent généralement qu'une partie des dimensions nécessaires à une détection complète des anomalies. DioT [Nguyen 2019] s'appuie ainsi sur des RNN de type Gated Recurrent Unit (GRU) pour modéliser les dépendances temporelles, tandis que VFL [Wei 2024] exploite GraphSAGE [Hamilton 2018] afin de capturer les relations structurelles entre entités. Dans ce contexte, le FL apparaît davantage comme une extension permettant de rendre la détection scalable et réaliste d'un point de vue opérationnel que d'une méthode de détection à part entière.

Dans la littérature des systèmes distribués, on retrouve également de nombreuses approches fondées sur l'analyse de logs systèmes, telles que [Liu 2010, Tan 2023]. Toutefois, ces méthodes présentent plusieurs limitations. Bien qu'elles permettent d'observer l'activité du système à un niveau relativement élevé, les logs constituent une représentation indirecte et partielle des événements réseau. Ils dépendent fortement des implémentations logicielles et peuvent contenir du bruit, des informations manquantes ou des incohérences. Ainsi, certains comportements anormaux peuvent ne jamais être journalisés, ce qui limite intrinsèquement la capacité de détection. Une piste complémentaire consiste à combiner différentes sources d'observation, comme proposé par Bogatinovski et al. [Bogatinovski 2021], en fusionnant logs et traces afin d'enrichir la représentation des événements. Cependant, cette approche introduit également une complexité supplémentaire dans la gestion et la corrélation des données, et peut conduire à une surcharge informationnelle.

Enfin, une part importante des travaux de la littérature s'intéresse aussi aux GNN [Protogerou 2021, Pazho 2024] et plus spécifiquement sur des approches dynamiques [Liu 2023]. Cette convergence entre les domaines de la 5G et des systèmes distribués confirme que la modélisation par graphes constitue un cadre particulièrement pertinent pour capturer les interactions complexes en intégrant à la fois les dimensions structurelles et temporelles des échanges.

1.2.4 Synthèse

L'analyse de la littérature sur la détection d'anomalies réseau met en évidence que les approches traditionnelles ne sont pas adaptées à notre problématique, principalement en raison d'un niveau de granularité trop élevé qui empêche la capture de comportements subtils. En se focalisant sur des travaux spécifiquement dédiés à la 5G, on constate par ailleurs qu'aucune approche ne propose à ce jour un formalisme unifié permettant de prendre en compte simultanément les trois dimensions de détection définies dans cette thèse. Néanmoins, plusieurs éléments

issus de l'état de l'art convergent vers des directions prometteuses. En particulier, des travaux tels que GSAD, ainsi que les approches issues des systèmes distribués, montrent que les représentations par graphes et les GNN font désormais l'objet d'un consensus croissant. Ces modèles apparaissent comme des outils particulièrement adaptés pour représenter la structure complexe des interactions et pourraient constituer une base solide pour répondre à notre problématique.

1.3 Méthodologie de formalisation

Nous avons vu que le formalisme en graphe proposé par GSAD consiste à regrouper l'ensemble du contenu des paquets dans un bloc monolithique associé à une arête, ce qui limite fortement la capacité à exploiter correctement la dimension sémantique. Dans ce contexte, nous proposons une représentation plus fine du trafic, en décomposant le contenu des paquets en nœuds indépendants correspondant chacun à un paramètre différent. Nous détaillons à présent notre méthodologie de formalisation. Nous commencerons par l'extraction des caractéristiques à partir du trafic de contrôle, puis nous présenterons leur transformation en graphes.

1.3.1 Extraction des features depuis la payload applicative

Comme notre objectif est principalement d'analyser le contenu applicatif des paquets, nous extrayons la payload applicative pour chaque paquet. Les seules informations non applicatives conservées concernent la couche IP, utilisées comme identifiant afin de discriminer les différentes entités du réseau. Cette approche n'est pas idéale, car elle est sensible au *spoofing*. Toutefois, dans un cadre opérationnel, ces adresses pourraient être remplacées par des identifiants plus robustes, tels que des empreintes ou des hash, qui pourraient représenter les entités de manière unique.

Pour récupérer le contenu des payloads, nous utilisons *Scapy*, qui permet de parser les paquets réseau et de générer, pour chacun d'eux, une représentation sous forme de dictionnaire associant les noms des paramètres à leurs valeurs respectives.

Une des problématiques du trafic applicatif est la présence d'imbrication dans le contenu des données de la payload. En effet, les paquets HTTP/2 contiennent couramment des structures JSON avec des données hiérarchisées. Conserver explicitement le chemin d'imbrication dans une modélisation en graphe nécessiterait d'introduire de nouveaux types de nœuds et donc de diluer l'information produite par le contenu lui-même. Or, même si l'imbrication est utile pour différencier des paramètres, elle reste secondaire par rapport aux valeurs elles-mêmes.

Pour gérer ce problème, nous avons donc choisi d'aplatir les dictionnaires extraits des payloads en construisant le nom de chaque paramètre de feuille comme la concaténation de ses clés précédentes, séparées par des délimiteurs. On obtient ainsi des identifiants de paramètres composés, par exemple `http2.headers.content-type`, permettant de désigner le champ `content-type` d'un paquet HTTP/2. Cette approche permet de conserver l'information

d'imbrication en tant que nom de paramètre tout en maintenant une structure suffisamment simple pour notre formalisation en graphe.

1.3.2 Transformation en graphe

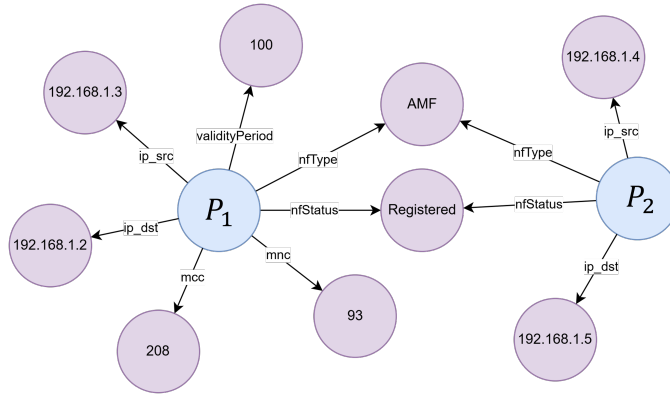


Figure 1.4: Exemple de transformation de paquets en sous-graphes. Les nœuds mauves représentent les valeurs des paramètres, tandis que les nœuds bleus représentent les paquets eux-mêmes.

Au lieu de représenter un paquet comme une arête entre deux entités, nous le modélisons comme un sous-graphe, où chaque paramètre est un nœud. Chacun de ces nœuds paramètre est relié à un nœud central qui représente le paquet. Chaque nœud paramètre contient la valeur du paramètre, tandis que l'arête reliant ce nœud au nœud central contient l'information du nom du paramètre correspondant. Lors de la création d'un sous-graphe représentant un paquet, de nouveaux nœuds paramètres sont créés seulement s'il n'existaient pas encore dans le graphe global. Dans le cas contraire, le nœud existant est réutilisé. Ce mécanisme permet de relier progressivement les différents sous-graphes entre eux à travers leurs paramètres partagés. La figure 1.4 illustre le processus de transformation de deux paquets en deux sous-graphes. Ces deux paquets partagent certains paramètres et leurs sous-graphes respectifs se retrouvent donc connectés.

Dans notre formalisme, la réception de nouveaux paquets ne peut entraîner que l'ajout de nouveaux nœuds et d'arêtes, sans jamais modifier ni supprimer ceux déjà présents. De plus, nous avons seulement deux types de nœuds : paramètres et centraux. Ces derniers ne peuvent jamais être reliés directement à d'autres nœuds du même type et notre formalisme est donc un graphe biparti purement additif.

Cette décomposition permet de connecter des attributs issus de paquets différents en s'appuyant sur des caractéristiques communes, plutôt que seulement sur l'identité des participants. Une telle représentation offre un niveau de granularité très fin et permet de capturer des interactions plus riches entre les caractéristiques des paquets. Enfin, cette approche permet également d'identifier précisément les paramètres à l'origine d'un comportement anormal et se révèle donc,

par construction, hautement interprétable.

Nous allons maintenant analyser la représentation des anomalies de chaque dimension une fois adaptés à notre formalisme.

1.3.2.1 Anomalies sémantiques

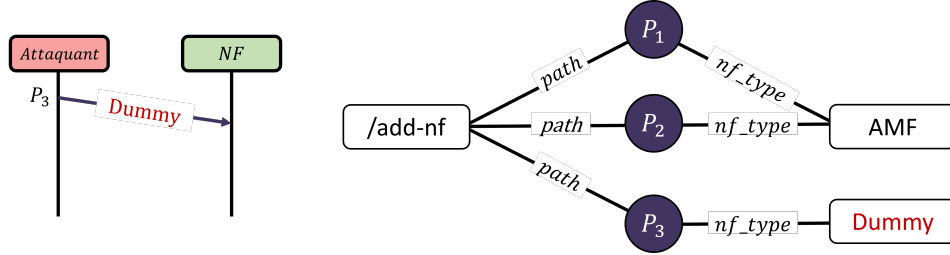


Figure 1.5: Anomalie sémantique et sa représentation sous forme de graphe.

La figure 1.5 illustre une anomalie sémantique dans laquelle un champ inhabituel apparaît. Dans le cas présent, il s'agit du paquet P3 dont la valeur du champs *nf-type* n'existe pas en 5G. Cette anomalie peut être détectée en le comparant aux autres paquets de même nature. On observe par exemple que les paquets P1 et P2, qui sont, comme P3, des requêtes vers l'endpoint *add-nf*, possèdent des valeurs identiques pour le champ *nf-type*, alors que P3 s'en écarte. Un modèle de détection ayant appris la structure usuelle de ce type de paquet pourrait ainsi identifier l'anomalie en observant uniquement le sous-graphe associé à P3.

1.3.2.2 Anomalies topologiques

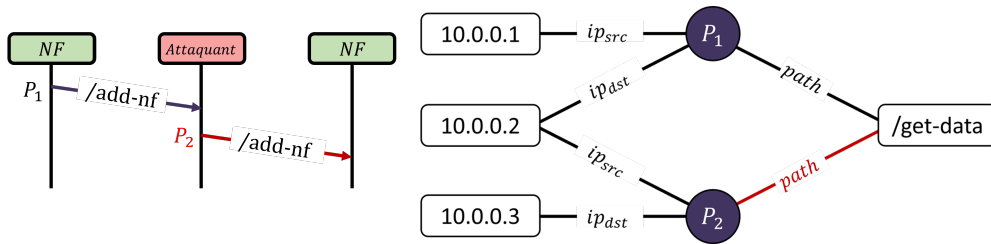


Figure 1.6: Anomalie topologique et sa représentation sous forme de graphe.

Sur la figure 1.6, on observe un exemple d'attaque de type Man-in-the-middle (MITM), où un paquet P1, initialement envoyé à une première adresse, génère l'envoi d'un nouveau paquet P2 identique (ici simplifié par le fait qu'il s'agisse du même endpoint). Ce nouveau paquet a comme source la destination du paquet précédent et comme destination une nouvelle adresse. Ce cas ne peut normalement pas survenir en 5G, car il n'existe aucun scénario légitime de redirection de trafic de ce type. Le paquet P2 est une copie de P1 et paraît donc légitime pris

individuellement. Il est ici nécessaire de prendre en compte le contexte de redirection afin de détecter l'anomalie.

1.3.2.3 Anomalies séquentielles

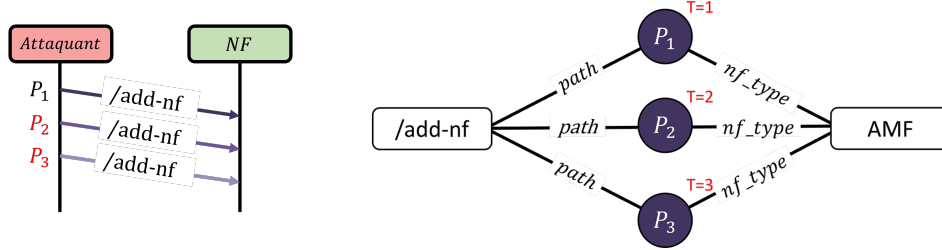


Figure 1.7: Anomalie séquentielle et sa représentation sous forme de graphe.

Enfin, sur la figure 1.7, on observe un exemple d'anomalie séquentielle. Dans ce cas, le contenu et les interactions des paquets sont normaux et la caractéristique anormale concerne uniquement leur ordre d'arrivée. Le problème est que le formalisme actuel ne modélise pas explicitement la dimension temporelle. Il serait possible de l'intégrer en ajoutant à chaque sous-graphe un nœud représentant l'horodatage du paquet, mais avec cette approche la dimension temporelle serait confondue avec les autres paramètres alors qu'elle est particulièrement importante. Il est donc nécessaire de proposer une modélisation permettant d'intégrer le temps de manière plus explicite.

1.3.3 Encodage des arcs et nœuds

Pour être exploitables par des modèles de détection, le contenu des nœuds et des arêtes doivent être encodés sous forme de vecteurs numériques. Pour les arêtes, le vecteur est construit à partir du nom du paramètre porté par l'événement, lequel est toujours de type textuel. L'espace des noms de paramètres possibles, le vocabulaire, est considéré comme statique car il dépend de l'implémentation de l'infrastructure 5G au moment de l'entraînement du modèle. Ainsi, si le dataset d'entraînement est suffisamment exhaustif, il est possible de couvrir la majorité des noms de paramètres susceptibles d'apparaître. Cependant, lorsqu'un paramètre inconnu est rencontré, deux cas peuvent être envisagés. Il peut s'agir d'une évolution de l'implémentation, auquel cas des anomalies peuvent être observées de manière légitime tant que le modèle n'a pas été ré-entraîné. Il peut également s'agir d'une tentative d'attaque, par exemple de fuzzing, qui exploite des appels API malformés et introduisant des noms de paramètres n'existant pas dans la requête. Dans ce cas nous avons une anomalie sémantique sur le nom des paramètres. Dans le cas des noms de paramètres, leur vocabulaire est donc ici statique et avec une grande valeur sémantique.

Pour les nœuds, le vecteur est construit à partir de la valeur du paramètre au lieu de son nom. Ces valeurs peuvent correspondre aussi bien à des variables

catégorielles, comme des méthodes HTTP, qu'à des identifiants uniques, comme des adresses IP. L'analyse de l'échantillon collecté dans notre dataset montre qu'environ 90% des valeurs de paramètres correspondent à des identifiants uniques. Ces identifiants sont générés au cours de l'exécution du système et ne peuvent donc pas être exhaustivement rencontrés lors de l'entraînement. Le vocabulaire des valeurs de paramètres est donc dynamique. En plus d'être indénombrables, les valeurs de paramètres ne possèdent pas de valeur sémantique intrinsèque. Leur sémantique est plutôt liée à leur utilisation dans les échanges et peut donc être inférée à travers l'analyse de la structure du graphe, comme illustré dans la figure 1.5. Les adresses IP, par exemple, n'ont pas d'intérêt intrinsèque en elles-mêmes, mais il est possible d'inférer si elles appartiennent à un attaquant en observant les paquets qu'il envoie et reçoit.

Dans nos représentations, seule l'information portée par les arêtes sera donc conservée explicitement. L'information associée aux nœuds pourra quant à elle être reconstruite implicitement par le GNN à travers les convolutions successives et l'agrégation des informations de voisinage.

1.4 Détection d'anomalies dans des graphes

La détection d'anomalies à l'aide de GNN constitue un domaine de recherche en pleine expansion, connaissant un développement significatif ces quinze dernières années. Nous introduirons dans cette section les principes fondamentaux de ces modèles et de leurs principales variantes. Nous examinerons ensuite l'état de l'art des GNN. Enfin, nous verrons comment adapter notre formalisme à sa variante la plus adaptée à notre problème, à savoir les graphes dynamiques continus.

1.4.1 Introduction aux GNN



Figure 1.8: Illustration du processus de convolution. La première figure représente le graphe à son état initial. Dans la deuxième, une première convolution est appliquée. On observe que les informations des nœuds feuilles sont agrégées et propagées à leurs voisins directs. Dans la troisième, une deuxième convolution est réalisée, et les informations précédemment propagées sont à nouveau diffusées jusqu'au nœud central du graphe. Après ces deux convolutions, le nœud central contient une représentation de son voisinage complet à distance 2.

Le fonctionnement des GNN est très proches de celui des CNN présentés précédemment. Dans les deux cas, le principe repose sur une agrégation locale de l'information, la principale différence étant le formalisme. Dans une image, cette agrégation s'effectue sur des voisinages de pixels, tandis que dans un graphe elle s'applique au voisinage des nœuds. Ces domaines sont donc très proches au point que certains travaux proposent de traiter une image comme un graphe, en considérant chaque pixel comme un nœud relié à ses voisins par un arc, sur lequel appliquer des GNN. Néanmoins, cette approche reste relativement marginale [Han 2022].

Que ce soit pour les CNN ou les GNN, l'idée fondamentale de ces modèles est que chaque itération du processus de convolution, appelé *Message Passing* pour les GNN, permet de diffuser l'information d'un individu à ses voisins directs. En procédant de manière récursive, chaque couche de convolution propage cette information à une distance croissante, augmentant progressivement le champ de réception des représentations. La figure 1.8 illustre ce processus de diffusion de l'information réalisé par deux convolutions successives.

En apprentissage sur graphes, on distingue généralement les approches transductives et inductives selon les informations disponibles au moment de l'entraînement. En mode transductif, le modèle connaît déjà toutes les données du dataset, y compris les nœuds à prédire et leur voisinage, mais leurs labels restent inconnus. À l'inverse, l'apprentissage inductif suppose que les nœuds à traiter pendant le test ne sont pas visibles durant l'entraînement. Le modèle doit donc apprendre des règles générales capables de se transférer à de nouvelles structures. Le choix entre ces deux paradigmes dépend fortement de la tâche considérée. Mais, de par leur nature évolutive, l'apprentissage inductif est souvent privilégié dans les travaux sur les graphes dynamiques, car il permet de généraliser à des nœuds et interactions apparaissant au cours du temps.

Parmi les tâches courantes dans l'état de l'art des graphes, on retrouve principalement la classification, la régression et le clustering. Ces tâches peuvent être appliquées à différents niveaux de granularité, que ce soit sur des nœuds, des arêtes ou des graphes entiers. La classification consiste à attribuer un label discret à un élément du graphe, tandis que la régression vise à prédire une valeur continue associée à celui-ci. Le *clustering*, quant à lui, a pour objectif de regrouper les éléments du graphe selon leurs représentations apprises. On retrouve également la tâche de prédiction de lien, qui consiste à estimer la probabilité d'existence d'une connexion entre deux entités du graphe. Cette tâche est particulièrement importante dans les graphes dynamiques, où l'objectif est d'anticiper l'évolution des interactions au cours du temps. C'est cette dernière famille de tâches qui nous intéresse particulièrement, puisque notre objectif est d'estimer, pour chaque valeur de paramètre, la probabilité qu'elle apparaisse et qu'elle soit donc considérée comme légitime.

L'étendue des problèmes pouvant être modélisés sous forme de graphes, ainsi que la diversité des tâches applicables, expliquent la croissance rapide de la littérature sur ce sujet.

1.4.2 Etat de l'art des GNN

Nous analyserons désormais l'état de l'art concernant les GNN. Notre étude s'attachera à l'évolution historique de ces approches, en commençant par les GNN statiques. Nous examinerons ensuite les graphes dynamiques et les différentes architectures de GNN adaptées à la manipulation de ces types de graphes. Enfin, nous analyserons les approches non-supervisées, indépendamment de la dimension temporelle, afin d'évaluer la capacité des GNN à traiter des données non étiquetées.

1.4.2.1 GNN traditionnels

Le domaine des GNN est relativement récent et s'est fortement développé au cours de la dernière décennie. Le concept de réseaux de neurones sur graphes a été formellement introduit pour la première fois en 2008 par Scarselli et al. [Scarselli 2009], qui ont proposé un framework pour l'apprentissage sur des données structurées en graphe, basé sur des mécanismes itératifs de propagation et d'agrégation d'états. Des travaux ultérieurs ont considérablement amélioré la praticabilité et la scalabilité de cette formulation initiale. En particulier, Kipf et Welling [Kipf 2017] ont introduit les Graph Convolutional Networks (GCN), qui simplifient le processus de propagation des messages en linéarisant l'agrégation de voisinage.

Dans leur formulation moderne, les GNN peuvent être vus comme réalisant des convolutions basées sur le voisinage, où l'information provenant des nœuds adjacents est agrégée afin d'enrichir la représentation de chaque nœud avec un contexte local. La plupart des architectures contemporaines de GNN suivent le paradigme de *message passing* proposé par Gilmer et al. [Gilmer 2017], composé de trois phases principales : la construction des messages à partir des embeddings des nœuds, l'agrégation des messages provenant des voisins, et la propagation (ou mise à jour) de l'information agrégée pour produire de nouvelles représentations des nœuds.

Sur la base du cadre Message Passing Neural Networks (MPNN) standard, plusieurs extensions ont été proposées. Par exemple, les Graph Attention Networks (GAT) [Veličković 2018] introduisent des mécanismes d'attention qui attribuent des poids adaptatifs aux nœuds voisins lors de l'agrégation. GraphSAGE [Hamilton 2018] permet un apprentissage inductif sur des nœuds jusqu'alors inconnus en échantillonnant un voisinage de taille fixe et en appliquant des fonctions d'agrégation apprenables telles que la moyenne, le pooling ou des agrégateurs basés sur LSTM. D'autres variantes notables incluent les Graph Isomorphism Networks (GIN) [Xu 2019] qui utilisent une agrégation par somme injective et des MLP pour mieux distinguer la structure globale du graphe.

Cependant, la plupart de ces architectures de GNN sont principalement conçues pour des graphes statiques. Or, nos données évoluent dans le temps et le graphe associé est donc dynamique. Nous nous intéressons donc aux modèles de graphes dynamiques, une classe encore plus récente de GNN. Parmi ces modèles, on distingue deux grandes catégories selon la granularité temporelle.

1.4.2.2 GNN Dynamiques

La première famille de graphes dynamiques, les Réseaux de Graphes à Temps Discret (DTGN), constitue l'approche la plus directe pour intégrer l'information temporelle. Dans ce cadre, l'évolution du graphe est modélisée comme une suite de captures de l'état du graph observés à intervalles réguliers. La dimension temporelle est ensuite prise en compte via des RNN appliqués à posteriori en complément du GNN. C'est notamment le cas de T-GCN [Zhao 2020] et EvolveGCN [Pareja 2019]. Cette formulation permet d'adapter relativement facilement des architectures de GNN classiques à un contexte temporel. DySAT [Sankar 2019] va un cran plus loin et propose un mécanisme d'attention temporelle où chaque nœud est associé à ses représentations historiques afin de capturer l'évolution de son contexte au cours du temps. Bien que ces approches aient l'avantage d'être relativement simples à mettre en œuvre, elles présentent plusieurs limites/ Premièrement, le fait de reposer sur une représentation par capture d'état implique de conserver l'état complet du graphe à plusieurs instants, ce qui engendre un coût mémoire élevé. Ensuite, la granularité temporelle est limitée par la fréquence des captures d'état, ce qui entraîne nécessairement une perte d'information. Enfin, en raison de l'aspect externe des modules de mémoire, ces méthodes peinent généralement à capturer les longues dépendances temporelles.

La seconde catégorie correspond aux réseaux de graphes en temps continu, qui opèrent à une granularité temporelle plus fine que les approches discrètes. Plutôt que de reposer sur des captures d'état fixes, ces modèles traitent des séquences d'événements correspondant à l'ajout, la suppression ou la modification de nœuds et d'arêtes dans le graphe. Cette modélisation événementielle permet d'intégrer directement les dépendances temporelles dans le modèle, plutôt que de les traiter de manière externe.

Parmi les modèles de graphes dynamiques en temps continu, on distingue généralement deux sous-familles : les modèles d'encodage temporel (*Temporal Embedding*) et ceux de voisinage temporel (*Temporal Neighborhood*). Les modèles d'encodage temporel ne stockent pas explicitement d'état associé aux nœuds et s'appuient principalement sur des architectures récurrentes ou attentionnelles pour capturer l'évolution temporelle à partir de la séquence des événements observés. Parmi eux, TGAT [Xu 2020] adapte le mécanisme de GAT [Veličković 2018] au contexte temporel en intégrant un encodage du temps directement dans le mécanisme d'attention.

La plupart des GNN dynamiques reposent sur des stratégies d'échantillonnage multi-saut afin de récupérer un voisinage temporel étendu. Ce processus est explicite dans la majorité des approches, mais peut également être implicite dans les modèles utilisant un RNN. CAWs [Wang 2022], au lieu de faire des parcours multi-saut classiques, propose une alternative fondée sur des parcours causaux qui exploite l'hypothèse de fermeture triadique, selon laquelle deux nœuds partageant un voisin commun ont davantage de chances d'être connectés. Cette hypothèse est cependant moins adaptée à notre cas d'étude basé sur un graphe biparti.

Plus récemment, DyGFormer [Yu 2023] simplifie cette approche en se limitant à un voisinage de distance 1, tout en ajoutant des informations de co-occurrence. Celles-ci sont calculées à partir de la fréquence d'apparition des nœuds dans les événements du voisinage temporel, puis injectées dans un Transformer. Les auteurs proposent également une implémentation unifiée facilitant la comparaison des modèles de graphes dynamiques. Comme DyGFormer, GraphMixer [Cong 2023] se limite également à un voisinage de distance 1. Les auteurs critiquent l'utilisation systématique de RNN et de mécanismes d'attention dans les graphes dynamiques, en expliquant que ces architectures ne sont nécessaires que pour agréger efficacement des dépendances multi-saut. En se limitant à un voisinage local, GraphMixer remplace ces mécanismes par des architectures de type MLP-Mixer, beaucoup plus simples et rapides à inférer.

À l'inverse des modèles d'encodage temporel, les modèles de voisinage temporel maintiennent un état latent associé à chaque nœud déjà observé, mis à jour au fil des événements. Cette approche est plus coûteuse en mémoire et en calcul, mais elle permet de conserver un historique condensé des interactions passées et de fournir davantage de contexte aux RNN lors du traitement de nouvelles interactions. Du côté des graphes dynamiques continus avec mémoire, on retrouve notamment DyRep [Trivedi 2018], qui distingue les événements de communication des événements d'association entre nœuds. Cette approche est particulièrement adaptée aux multigraphes dynamiques, mais elle est moins pertinente dans notre cas où il n'existe qu'une seule arête possible par paire de nœuds. JODIE [Kumar 2019] est quant à lui conçu pour la prédiction de liens sur graphes bipartis. Le modèle maintient pour chaque utilisateur et chaque item un embedding dynamique mis à jour par des RNN mutuellement récurrents, en complément d'une représentation statique. Enfin, TGN [Rossi 2020] propose une formalisation générale des graphes dynamiques continus avec mémoire. Le modèle introduit une architecture modulaire composée d'un mécanisme de mémoire, d'une fonction de mise à jour, d'un module de message passing et d'un encodeur temporel. De nombreux modèles existants peuvent ainsi être interprétés comme des cas particuliers de ce formalisme.

	Modèle	Attention	RNN	Mémoire	Parcours
Temporal Embedding	TGAT [Xu 2020]	Oui	Non	Non	multi-saut
	CAWs [Wang 2022]	Oui	Oui	Non	causal
	DyGFormer [Yu 2023]	Oui	Non	Non	1-hop
	GraphMixer [Cong 2023]	Non	Non	Non	1-hop
Temporal Neighborhood	DyRep [Trivedi 2018]	Non	Oui	Oui	multi-saut
	JODIE [Kumar 2019]	Non	Oui	Oui	multi-saut
	TGN [Rossi 2020]	Oui	Oui	Oui	multi-saut

Table 1.2: Comparaison des principaux modèles de graphes dynamiques en temps continu.

Une vue d'ensemble de l'apprentissage sur graphes dynamiques est proposée par Feng et al. [Feng 2024], qui présentent une synthèse mettant en évidence les différences conceptuelles entre les modèles dynamiques à temps discret et à

temps continu, ainsi qu'un benchmark étendu des principales méthodes de ces deux catégories. Le tableau 1.2 fournit une comparaison synthétique de ces modèles selon leurs mécanismes d'attention, leur utilisation de RNN et de mémoire, ainsi que leur stratégie d'échantillonnage de voisinage.

1.4.2.3 GNN non supervisé

L'ensemble des modèles précédemment énoncés sont conçus pour pouvoir être utilisés sur des tâches de prédiction de liens, qui relèvent de l'apprentissage auto-supervisé. Ce cadre est généralement considéré comme une sous-catégorie de l'apprentissage non supervisé, dans lequel des pseudo-labels sont construits automatiquement à partir des données brutes. Dans le cas des graphes, cela consiste à opposer des arêtes positives, correspondant aux interactions réellement observées dans le graphe, à des arêtes négatives, générées artificiellement à partir de paires de nœuds n'ayant jamais été connectées. Ces arêtes sont encodées puis passées dans une couche dense qui prédit la probabilité d'existence de l'arête. La prédiction obtenue est ensuite comparée à la vérité afin de calculer la fonction de perte. Néanmoins, la génération d'arcs négatifs pertinents constitue un défi difficile. Il est en effet, à la fois possible de produire des arcs négatifs qui pourraient en réalité être légitimes, ou au contraire de ne pas couvrir correctement la diversité des comportements anormaux lorsque ceux-ci sont échantillonnés de manière aléatoire. Ce problème est d'autant plus marqué dans des contextes tels que le nôtre, où les sous-graphes sont fortement structurés et contraints.

L'autre approche non-supervisée couramment utilisée pour la détection d'anomalies consiste à formuler le problème comme une tâche de reconstruction des données. Pour ce faire, on utilise un autoencodeur qui prend en entrée un graphe et génère une représentation latente compressée à partir duquel il va essayer de recréer l'entrée d'origine. Après avoir été entraîné sur des données ne contenant pas d'anomalies, l'autoencodeur est capable de reconstruire correctement les points considérés comme normaux, mais devrait normalement échouer davantage sur des données atypiques qu'il n'a jamais observées auparavant. L'erreur de reconstruction peut alors être utilisée comme score d'anomalie. Plus cette erreur est élevée, plus le point est susceptible d'être anormal. C'est notamment le principe utilisé par DONE [Bandyopadhyay 2020], qui repose sur deux autoencodeurs parallèles. Le premier encode la structure des liens du graphe, tandis que l'autre encode les attributs des nœuds. Les auteurs proposent également AdONE [Bandyopadhyay 2020], une extension basée sur l'apprentissage adverse permettant de construire des embeddings plus robustes aux valeurs aberrantes. GraphMAE [Hou 2022] utilise également une architecture de type autoencodeur. Leur contribution principale consiste à appliquer un double masquage. Un premier masque est appliqué sur les attributs des nœuds avant l'encodage, puis un second est appliqué dans l'espace latent après l'encodage. Dans leur cas, l'encodeur est un GNN, tandis que le décodeur est un MLP. Cette stratégie de masquage vise à renforcer la robustesse des représentations apprises en forçant le modèle à inférer des informations

manquantes à différents niveaux de représentation. STGMAE [Zhang 2019] est presque identique mais enrichit l'entrée du GNN par une couche d'encodage spatio-temporel. Toutefois, ces approches restent limitées au cadre des graphes statiques et ne prennent pas directement en compte l'évolution temporelle de la structure du graphe. DyGIS [Jiao 2024] adapte justement GraphMAE aux graphes discrets dynamiques mais, à notre connaissance, aucun travail de l'état de l'art ne propose encore d'étendre les méthodes de reconstruction au cas des graphes dynamiques en temps continu non supervisé.

Parmi les deux approches d'apprentissage non-supervisées présentées, la prédiction d'arcs via l'auto-supervision semble plus facilement équilibrable, étant donné la nature binaire de la tâche. Cependant, la création d'arcs négatifs qui est nécessaire à l'auto-supervision est sujette à l'introduction de biais. En revanche, l'approche de reconstruction est moins susceptible à ces biais mais est plus difficile à guider.

1.4.3 Formalisation adapté à l'utilisation de GNN dynamiques

Pour considérer la dimension temporelle et détecter les attaques d'inondation ou à plusieurs étapes, il faut intégrer l'information temporelle à notre détection et donc considérer les graphes dynamiques. Une solution serait d'utiliser les graphes dynamiques discrets et c'est d'ailleurs l'approche qu'emploie GSAD [Wang 2023]. Toutefois, nous avons vu que cette approche est peu adaptée, car elle est coûteuse, entraîne des pertes d'information et est peu adaptée aux périodes d'observation étendues.

Nous nous intéressons donc davantage aux graphes dynamiques continus, tels qu'introduits dans le cadre de TGN [Rossi 2020]. Dans cette approche, plutôt que de représenter le graphe comme un objet statique décrivant un état à un instant donné, celui-ci est modélisé comme une séquence d'événements qui le construit progressivement.

$$\mathcal{G} = \{e_1, e_2, \dots, e_n\}$$

où e_i désigne l'événement apparaissant à la i -ème position dans la séquence.

Ces événements correspondent généralement à des ajouts, suppressions ou modifications de nœuds et d'arêtes. Toutefois, notre cadre est plus simple car notre graphe est purement additif. Chaque arête qui serait ajoutée dans un graphe traditionnel est représentée comme un événement défini par cette arête ainsi que par les deux nœuds qu'elle relie.

$$e_i = (u, v_i, x_i),$$

où $u \in \mathcal{V}_c$ désigne un nœud central, $v_i \in \mathcal{V}_f$ un nœud de paramètre contenant la valeur de ce dernier et $\mathbf{z}_i \in \mathbb{R}^d$ le nom du paramètre contenu dans l'arête.

Cette formulation permet une représentation beaucoup plus fine, dans laquelle aucune information temporelle n'est perdue. La figure 1.9 illustre la transformation

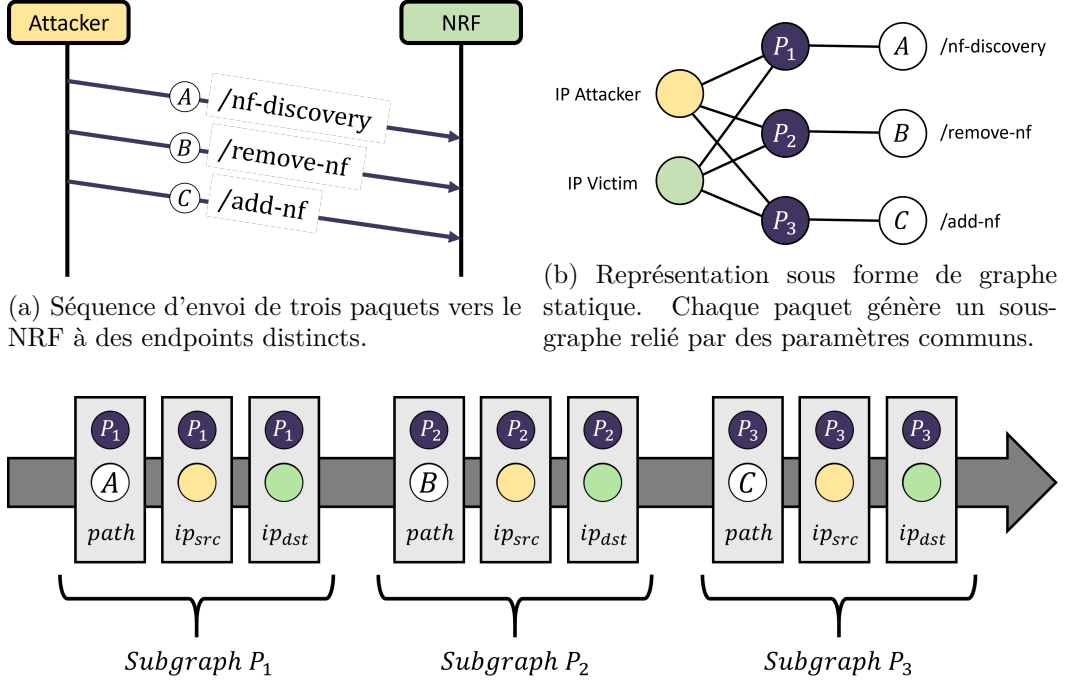


Figure 1.9: Différentes représentations d'une même séquence d'échanges entre un attaquant et le NRF : diagramme de séquence, graphe statique, puis graphe dynamique continu.

d'une forme de graphe statique à celle d'un graphe dynamique continu. On observe que, pour les trois paquets, neuf arcs sont générés dans le graphe statique, lesquels se traduisent ensuite en autant d'événements dans le graphe continu.

1.4.4 Fonctionnement détaillé des GNN dynamique

Dans leurs travaux, TGN [Rossi 2020] proposent un cadre de formalisation des GNN dynamiques, permettant de représenter les travaux de l'état de l'art comme des spécifications de leur modèle. Nous nous appuyerons sur ce cadre pour présenter et comprendre le fonctionnement d'un GNN dynamique. La figure 1.10 illustre les quatre modules principaux définis par TGN [Rossi 2020] : un module de mémoire, un module d'agrégation de messages, un module de mise à jour de la mémoire et un module d'inférence.

1.4.4.1 Création des messages

La première étape d'un GNN consiste à construire un message contenant l'ensemble des informations associées à un événement du graphe dynamique.

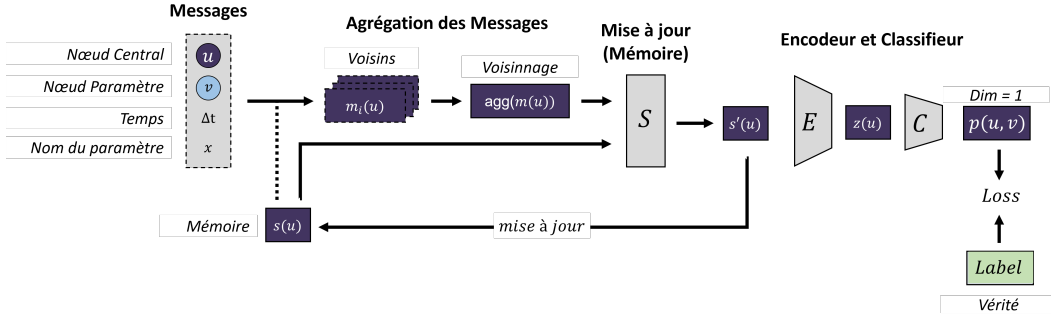


Figure 1.10: Vue d'ensemble de l'architecture générale d'un GNN Dynamique Continu, illustrant les quatre modules principaux : mémoire, agrégation de messages, mise à jour de la mémoire et inférence.

$$m_i = msg(s(u), s(v_i), x_i, \Delta t)$$

Comme expliqué dans la section 3.3.3, la valeur des nœuds paramètres ne nous intéressent pas. Les vecteurs associés aux nœuds centraux $s(u)$ et aux nœuds paramètres $s(v_i)$ sont donc initialisés comme des vecteurs nuls. Si ces nœuds ont déjà été impliqués dans des événements précédents, leurs représentations qui a déjà subi une convolution est alors réutilisées. Le vecteur associé à l'arête x_i est quant à lui déterminé à partir du nom du paramètre porté par l'événement. Dans notre cas, ce nom de paramètre correspond à un token textuel et il est donc nécessaire de l'encoder au préalable sous forme d'un vecteur numérique. La méthode utilisée pour cet encodage est présentée dans la section suivante. Il existe diverses méthodes pour définir l'intervalle temporel Δt mais la méthode la plus fréquemment employée consiste à calculer la différence temporelle entre l'échantillon et le dernier paquet encodé. Ce dernier est représenté sous forme de vecteur de dimensions identiques aux autres composantes du système. Un paramètre ajustable permet alors de moduler la granularité de cette mesure.

1.4.4.2 Agrégation des messages

L'étape d'agrégation consiste à considérer l'ensemble des voisins du nœud source en cours de traitement, c'est-à-dire tous les événements m_i précédant un temps t dans lesquels ce nœud apparaît. On obtient ainsi une collection de messages qu'on agrège ensuite afin de produire un vecteur unique représentant le voisinage de distance 1 du nœud considéré.

$$\bar{m}_i(t) = \text{agg}(m_i(t_1), \dots, m_i(t))$$

Cette agrégation peut être réalisée de différentes manières. TGN [Rossi 2020] propose plusieurs stratégies, parmi lesquelles l'utilisation de RNN, d'un mécanisme d'attention, une moyenne des messages ou bien de simplement conserver l'événement

le plus récent.

1.4.4.3 Mise à jour de la mémoire

Afin de prendre en compte la dimension temporelle, les modèles dynamiques de type *Temporal Neighborhood* disposent d'un module de mémoire, absent des modèles *Temporal Embedding*. Ce module combine le vecteur agrégé avec l'historique des représentations précédentes des éléments constitutifs du message. Cet ensemble est ensuite transmis à un RNN, qui joue le rôle de module de mémoire apprenable. Ce module produit un nouveau vecteur représentant à la fois le message et son historique.

$$s(u) = mem(x_i, s(u))$$

Cette nouvelle représentation est utilisée pour les modules futurs et remplace également l'ancienne valeur stockée en mémoire. TGN [Rossi 2020] propose notamment d'utiliser des architectures de type Long Short-Term Memory (LSTM) ou GRU pour implémenter ce module de mémoire.

1.4.4.4 Encodeur et Classifieur

L'encodeur est le module chargé de calculer la représentation des nœuds à partir de leur mémoire et des interactions temporelles. Plusieurs variantes peuvent être utilisées :

- **Identity (id)** : l'encodage correspond directement à la mémoire du nœud.
- **Time projection (time)** : la mémoire est modulée en fonction du temps écoulé depuis la dernière interaction, comme dans JODIE [Kumar 2019].
- **Temporal Graph Attention (attn)** : plusieurs couches de graph attention agrègent l'information provenant du voisinage temporel du nœud. À chaque couche, les voisins temporels sont récupérés puis combinés via un mécanisme de multi-head attention.
- **Temporal Graph Sum (sum)** : version plus simple et rapide où les informations des voisins temporels sont agrégées par somme puis projetées linéairement.

Pour les modèles *attn* et *sum*, un encodage temporel de type *Time2Vec* est utilisé afin de représenter les écarts temporels entre interactions. Ces méthodes permettent également de limiter le problème de *staleness* en mettant à jour les représentations grâce aux informations des voisins récents.

1.4.5 Adaptation à notre cas d'utilisation

Pour mieux répondre à notre problématique, nous proposons deux ajustements justifiés par notre cadre d'utilisation. Dans un premier temps, nous supprimons l'information temporelle explicite des messages. Dans un second temps, nous adaptons le mécanisme d'agrégation standard en une version spécifique, conçue pour notre structure bipartite.

Le premier ajustement consiste à supprimer l'information temporelle explicite du modèle. Concrètement, cela se traduit par le retrait du terme Δt dans la formulation des messages. Dans notre contexte, l'information temporelle est déjà implicitement capturée par les séquences d'interactions lorsqu'elle est traitée par le RNN. La mention explicite du terme Δt est principalement pertinente pour identifier d'éventuelles pauses ou délais dans le trafic. Toutefois, cette information ne constitue pas un indicateur suffisamment représentatif d'une attaque et induit souvent, au contraire, des biais de surapprentissage. Pour cette raison, nous avons choisi de retirer cette composante temporelle de la formulation des messages.

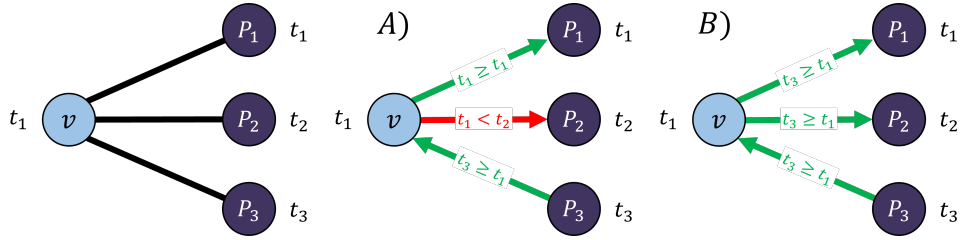


Figure 1.11: Représentation du problème lié au parcours multi-saut dans notre graphe biparti. La première figure représente le graphe avec les arrivées temporelles des paquets. Les figures suivantes représentent les deux méthodes possibles de propagation de l'information du paquet P3. (A) La méthode classique de propagation multi-sauts utilise successivement le timestamp des nœuds intermédiaire. (B) Notre approche conserve le timestamp du nœud initial, et permet donc de récupérer correctement les voisins multi-sauts.

Le deuxième ajustement concerne la construction des voisinages multi-sauts. Dans un GNN dynamique classique, un nœud ne peut normalement agréger que des voisins apparus avant lui. Cependant, dans notre graphe biparti, les nœuds paramètres sont réutilisés et conservent leur horodatage de première apparition. Cela crée un problème lors de l'exploration multi-sauts, car l'horodatage des nœuds paramètres sera antérieur à celui de la majorité des paquets auxquels il sera lié. Ces nœuds centraux seront donc considérés à tort comme inaccessibles lors de l'agrégation du voisinage. Pour résoudre ce problème, nous conservons l'horodatage du nœud initial lors de l'exploration du voisinage, plutôt que récupérer successivement celui des nœuds intermédiaires, ce qui permet de conserver un voisinage temporel cohérent. La figure 1.11 présente un exemple d'une telle situation ainsi que les deux approches de propagation.

Ces ajustements répondent directement aux spécificités de notre cas d'usage et

ont donc été intégrés à notre approche. La suppression de l'information temporelle explicite permet de limiter les risques de surapprentissage liés aux irrégularités du trafic, tandis que la modification du mécanisme de propagation garantit une exploration multi-sauts cohérente dans le contexte particulier de notre graphe biparti.

1.4.5.1 Problèmes ouverts

D'autres pistes d'ajustement ont également été envisagées au cours de cette thèse, mais n'ont finalement pas été intégrées à l'approche proposée. Elles restent néanmoins intéressantes à mentionner, car leur considération est légitime et soulève plusieurs questions pertinentes quant à notre formalisation.

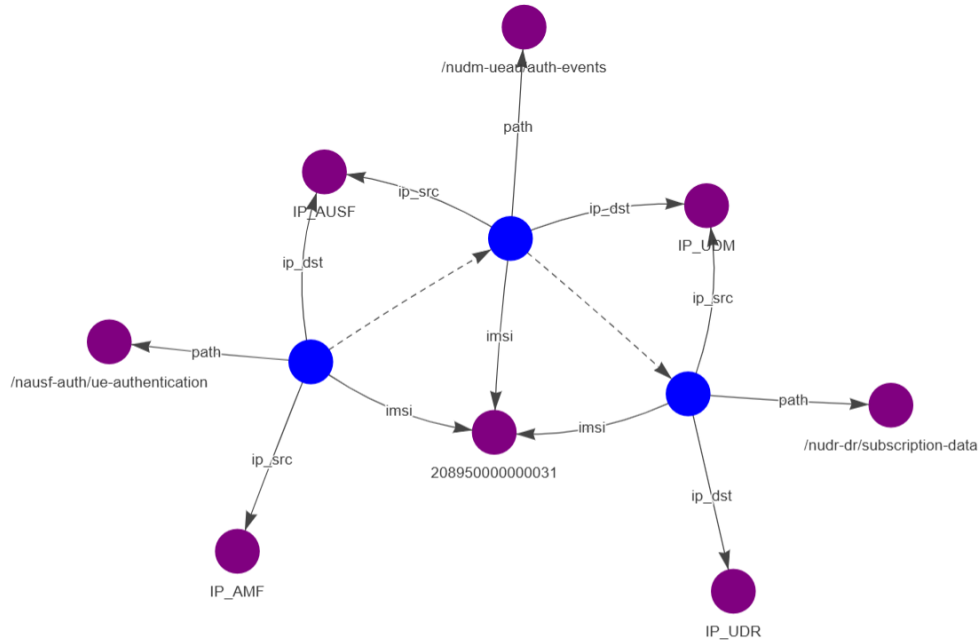


Figure 1.12: Représentation sous forme de graphe d'une procédure composée de trois paquets. Les arcs orientés en pointillés représentent un nouveau type de liaison entre nœuds central permettant de représenter l'ordre temporel des échanges.

Dans un réseau 5G, le déclenchement d'une procédure engendre la génération de séquences ordonnées de messages. Il peut être envisagé de représenter explicitement ces séquences ainsi que leur ordre au sein du graphe au moyen d'une formalisation dédiée. Une approche possible consiste à orienter le graphe et à introduire un type de liaison spécifique pour relier un nœud central à son successeur dans la séquence. La Figure 1.12 illustre l'ajout de dépendance par une flèche pointillée. Cette représentation présente l'avantage d'être aisément interprétable par un lecteur humain tout en rendant explicites les séquences et leur ordre d'exécution. Cependant, cette approche n'apporte pas d'information supplémentaire au modèle, dans la mesure où la structure séquentielle est déjà encodée via les graphes

dynamiques continus. Néanmoins, elle conserve un intérêt pour l’analyse humaine et peut ainsi être ajoutée a posteriori de l’inférence afin d’améliorer la lisibilité des résultats.

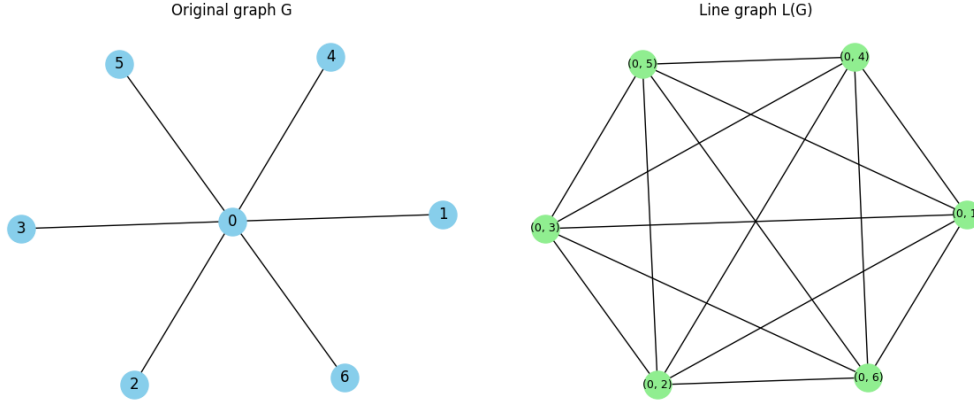


Figure 1.13: Illustration de la transformation d’un graphe classique (à gauche) en *line graph* (à droite). Dans cette représentation, les arêtes du graphe initial deviennent les nœuds du *line graph*, tandis que les relations entre arêtes sont représentées par de nouvelles connexions.

EGAT [Chen 2021] propose de donner plus d’importance à l’information portée par les arêtes en réalisant une agrégation sur ces dernières au lieu d’une agrégation sur les nœuds comme cela est fait classiquement. Pour ce faire, ils convertissent leur graphe en *line graph* où les nœuds correspondent aux arêtes du graphe original, tandis que les arêtes traduisent les relations entre les nœuds. La Figure 1.13 illustre cette transformation. Néanmoins, dans notre cas d’étude, les graphes considérés sont fortement connectés et comportent un grand nombre d’arêtes. Or, la construction du *line graph* entraîne une augmentation significative de la complexité structurelle, puisque chaque arête du graphe initial devient un nœud supplémentaire. Cette transformation ne passe donc pas à l’échelle dans notre configuration et conduit rapidement à une explosion de la complexité.

Ces différentes approches montrent que plusieurs extensions de la formalisation initiale sont envisageables. Néanmoins, bien qu’elles présentent des avantages évidents, elles soulèvent des problèmes structurels et computationnels qui empêchent leur intégration à notre formalisation.

1.5 Encodage des paramètres contenus dans les arcs

Comme mentionné dans la section 3.4.4.1 relative à la construction des messages, il est nécessaire d’encoder les informations contenues dans les arcs. Les paquets réseau 5G comportent un très grand nombre de paramètres, ce qui conduit à un espace d’entrée de très grande dimension. Nous verrons dans cette section que cette forte dimensionnalité ne constitue pas nécessairement un problème dans le cadre d’une

approche supervisée. En revanche, si l'objectif est de réaliser une détection non supervisée afin d'identifier des attaques *zero-day*, il devient nécessaire de recourir à des techniques d'encodage permettant de réduire la dimension de l'espace d'entrée. À cette fin, nous nous intéressons aux approches issues du Traitement Automatique du Langage Naturel (NLP). Nous analyserons leur utilisation dans l'état de l'art, puis étudierons leur adaptation à notre cas d'usage ainsi que les problématiques qu'elles soulèvent.

1.5.1 Apprentissage supervisé et non supervisé

L'apprentissage supervisé et l'apprentissage non supervisé soulèvent des problématiques différentes en matière d'encodage des paramètres. Nous commençons donc par présenter ces deux approches, avant d'examiner les implications de chacune sur la stratégie d'encodage des paramètres.

1.5.1.1 Apprentissage supervisé

Le principe de l'apprentissage supervisé consiste à entraîner un modèle sur un dataset étiqueté contenant à la fois des exemples bénins et malveillants. L'objectif est d'apprendre à distinguer ces deux classes afin de pouvoir ensuite effectuer cette distinction sans étiquettes. Ce type d'apprentissage se limite aux attaques connues, mais il offre généralement de meilleures performances. Cependant, comme l'ensemble des attaques à détecter est défini à l'avance, il est possible de sélectionner un sous-ensemble de paramètres pertinents et de réduire ainsi la dimension du problème. En considérant un sous-ensemble suffisamment restreint, il devient alors possible d'utiliser de simples techniques de *one-hot encoding* pour encoder les paramètres.

1.5.1.2 Apprentissage non supervisé

L'apprentissage non supervisé consiste à apprendre à un modèle le comportement normal d'un système en l'entraînant uniquement sur des données bénignes. Les anomalies sont alors détectées à partir de leur écart à cette notion de normalité. Ce type d'apprentissage permet de détecter des attaques inconnues, mais il est généralement plus difficile à mettre en œuvre. En effet, puisque l'objectif est notamment de détecter des attaques de type *zero-day*, il n'est pas possible de savoir à l'avance quels paramètres seront pertinents. Il est donc nécessaire de considérer l'ensemble des paramètres potentiellement observables, y compris ceux qui ne sont pas pertinents pour les attaques connues, ce qui introduit un bruit important dans les données.

De plus, la prise en compte de l'ensemble des paramètres potentiellement observables conduit à un espace d'entrée de très grande dimension, susceptible d'entraîner le phénomène de *malédiction de la dimension*, caractérisé par une dégradation des capacités de généralisation lorsque la dimension de l'espace d'entrée devient trop importante. Par conséquent, il est nécessaire de réduire la dimension

de cet espace à l'aide de techniques issues du NLP, afin de rendre les données exploitables par les modèles de GNN présentés précédemment.

1.5.2 Tour d'horizon des techniques de NLP

Les approches issues du NLP sont traditionnellement utilisées pour des tâches telles que la traduction automatique, la classification de documents, la détection d'intention ou encore la reconnaissance d'entités nommées. Dans ces applications, les modèles sont entraînés à apprendre une représentation sémantique des données textuelles, sous la forme d'un espace latent structuré dans lequel une proximité géométrique reflète une proximité sémantique. Dans cette thèse, nous cherchons à exploiter directement la structure de l'espace latent afin d'associer à n'importe quel paramètre une représentation vectorielle pertinente.

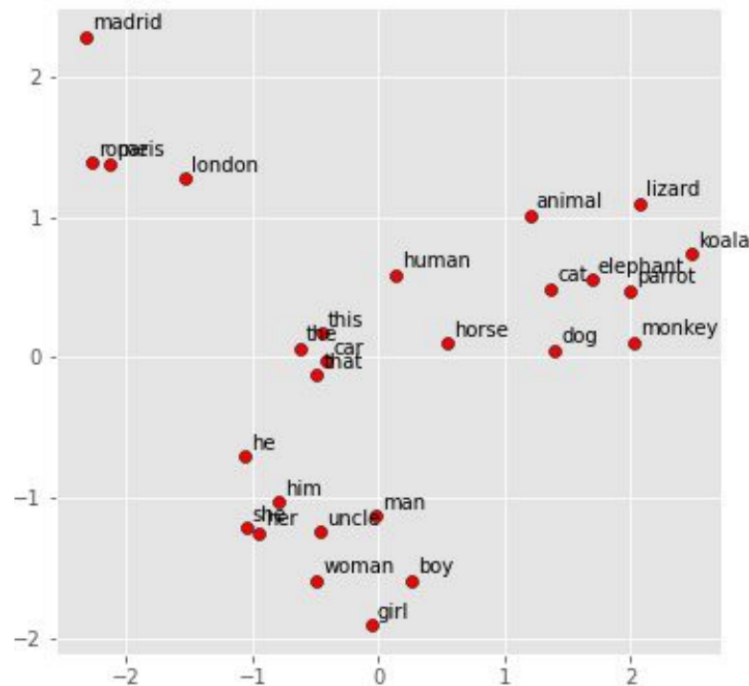


Figure 1.14: Projection en deux dimensions d'un espace latent représentant un vocabulaire anglais, obtenue par Analyse en Composantes Principales (PCA).

Pour évaluer la qualité de cette structuration, deux approches complémentaires sont généralement utilisées et reposent sur une interprétation humaine. La première consiste à examiner, au cas par cas, les voisins les plus proches d'un mot donné dans l'espace d'embedding. La seconde consiste à projeter cet espace de haute dimension dans un espace à deux dimensions afin d'en permettre une visualisation globale. Les méthodes les plus couramment utilisées pour cela sont la PCA et le t-SNE, qui représentent respectivement une projection linéaire conservant la structure globale, et une projection stochastique visant à préserver les similarités locales. La figure

1.14 illustre un exemple de projection PCA d'un vocabulaire générique encodé avec FastText [Bojanowski 2017] que nous allons présenter ensuite.

Notre objectif est d'encoder les mots de manière individuelle. L'approche la plus simple consiste à utiliser un encodage one-hot, c'est-à-dire associer chaque mot à une dimension d'un vecteur orthogonal. Le fait d'attribuer des représentations uniques à chaque mot laisse ensuite au GNN la responsabilité d'en apprendre la sémantique. Un inconvénient de cette méthode est qu'elle produit des embeddings sans sémantique qui pourrait pourtant faciliter l'apprentissage du GNN. De plus, comme la construction de vecteurs orthogonaux impose une dimension de l'espace d'embedding égale à la taille du vocabulaire, cette approche n'est pas possible sur des espaces de grande dimension et est donc réservée, dans notre cas, à l'apprentissage supervisé.

Afin de pallier ce problème, des techniques comme le hashing trick projettent les termes dans un espace de dimension fixe à l'aide d'une fonction de hachage. Cette approche permet de contrôler la taille des représentations indépendamment du vocabulaire, mais introduit des collisions. Ainsi, plusieurs termes distincts peuvent obtenir des encodages identiques, ce qui peut induire des proximités artificielles dans l'espace vectoriel. Le hashing trick et le one-hot encoding ont l'avantage d'être simples et rapides à grande échelle, mais ne capturent pas la dimension sémantique des mots qu'ils encodent.

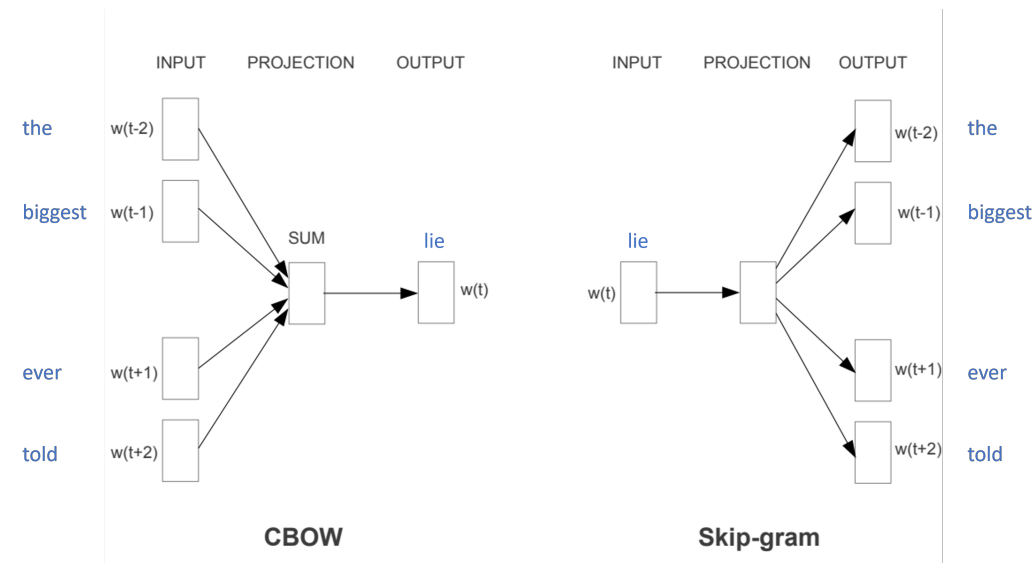


Figure 1.15: Illustration de l'architecture de Word2Vec à travers les modèles Continuous Bag of Words (CBOW) et Skip-Gram, appliqués au mot cible "lie" dans la phrase "The biggest lie ever told".

Les approches de deep learning permettent justement d'apprendre la sémantique des mots à partir de leur contexte d'utilisation. L'idée principale est qu'un mot peut être compris en observant les mots qui l'entourent dans une phrase ou un corpus. C'est ce que fait Word2Vec [Mikolov 2013] qui propose deux modèles, un CBOW

qui vise à prédire un mot à partir de son contexte et un Skip-Gram qui, de façon symétrique, vise à prédire les mots du contexte en connaissant un mot en entrée.

Néanmoins, comme il apprend sur des fenêtres de contexte locales, Word2Vec ne dispose pas d'une vision globale du corpus. Cette limitation est adressée par Global Vectors for Word Representation (GloVe) [Brochier 2019], qui apprend à minimiser la distance entre le produit scalaire de deux vecteurs et le logarithme de leur fréquence de co-occurrence dans le corpus. Cependant, GloVe présente des limites dans le cas de langues à morphologie et vocabulaire riche.

FastText [Bojanowski 2017] est justement une extension de Word2Vec qui cherche à découper un mot en sous-unités morphologiques. Contrairement à Word2vec, FastText ne considère pas un mot comme une entité atomique, mais le représente comme un ensemble de n -grammes de caractères. Cette segmentation permet de partager des informations entre mots présentant des structures similaires. Dans notre cas, cela permet par exemple de décomposer *servingNfId* en plusieurs sous-unités telles que *serving*, *Nf* et *Id*. Chacun de ces segments peut alors porter une information sémantique propre et être réutilisée pour représenter d'autres paramètres partageant des sous-parties communes. Cependant, une limite importante de cette approche réside dans le caractère statique des embeddings. Ici, chaque mot et n -gramme possède une représentation fixe qui est indépendante du contexte dans lequel il apparaît.

Or, un mot peut être polysémique et donc avoir une sémantique variable en fonction de son contexte d'apparition. Afin de prendre en compte cette ambiguïté sémantique, on a vu l'arrivée de travaux utilisant des architectures de type Transformer, qui permettent d'apprendre des représentations contextuelles dynamiques. Ces modèles, tels que GPT [Radford 2018] et BERT [Devlin 2019], utilisent des mécanismes d'attention pour modéliser les dépendances à long terme dans les séquences de texte, ce qui améliore significativement la qualité des représentations apprises. Le premier est basé sur un décodeur qui va analyser le contexte historique d'un mot pour prédire sa suite tandis que le second va prendre le contexte bidirectionnel et sera plus adapté à des tâches de compréhension du langage. Ces modèles sont la plupart du temps pré-entraînés sur de très grands corpus de texte, mais peuvent être adaptés à des tâches spécifiques par le biais d'une phase de fine-tuning. Cependant, leur complexité et leurs besoins en ressources computationnelles sont considérablement plus élevés que les modèles précédents.

Parmi les approches de NLP proposées, GPT, BERT et leurs variantes respectives figurent parmi les modèles obtenant généralement les meilleures performances. Néanmoins, il convient de souligner que ces architectures ont été conçues pour des données textuelles naturelles et reposent sur des hypothèses implicites liées à la structure du langage humain. Dans le cadre de cette thèse, le contexte considéré est sensiblement différent. Le vocabulaire manipulé est très spécifique, parfois constitué d'identifiants ou de chaînes non lexicalisées, et les paquets qui les contiennent diffèrent fortement de la structure d'une phrase naturelle. Dès lors, l'utilisation de modèles aussi complexes peut s'avérer disproportionnée au regard de la nature des données. Il apparaît donc nécessaire

d'évaluer dans quelle mesure ces architectures peuvent être adaptées à notre type de données, et d'identifier les modèles offrant le meilleur compromis entre expressivité, robustesse et coût computationnel dans un contexte aussi spécifique.

1.5.3 Mise en place de l'encodage sémantique

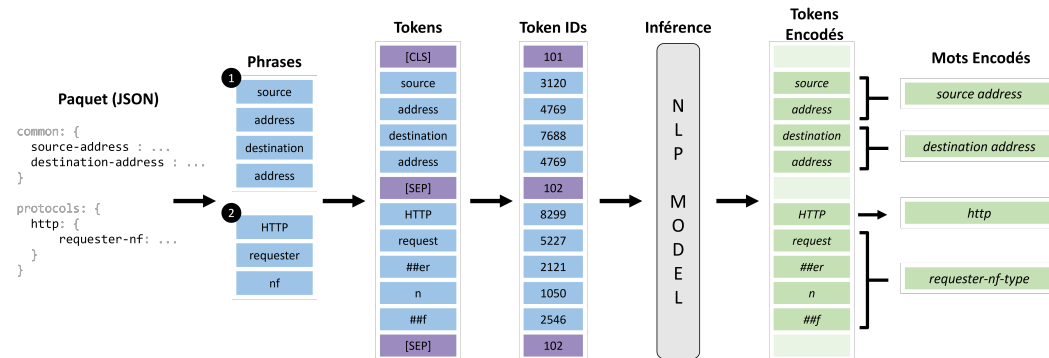


Figure 1.16: Représentation du processus d'encodage fondé sur le NLP. Les paramètres du paquet sont tout d'abord extraits puis transformés en séquences textuelles, lesquelles sont ensuite tokenisées. Ces tokens sont transmis à un modèle de NLP permettant d'obtenir leurs représentations encodées. Ces représentations sont finalement agrégées afin d'obtenir un encodage au niveau du mot.

La figure 1.16 illustre le processus d'application du NLP à notre problème afin d'obtenir des encodages associés aux mots. Dans un premier temps, nous exploitons le fichier JSON contenant les données extraites des paquets, que nous transformons en phrases. Pour cela, les chaînes de caractères sont nettoyées en supprimant les caractères spéciaux, tels que les tirets ou les points. Pour chaque couche du paquet, une phrase distincte est construite, et le nom du protocole est explicitement ajouté afin d'enrichir le contexte lexical. Le modèle considère ainsi l'ensemble du paquet comme une unité globale, tout en tirant parti de la séparation entre les phrases pour structurer l'information.

Ensuite, comme dans un pipeline NLP classique, une étape de tokenisation est appliquée. Celle-ci consiste à identifier des mots connus dans le vocabulaire et, lorsqu'un mot n'est pas reconnu, à le décomposer en sous-unités. Comme illustré sur la figure, le mot *requester* est par exemple segmenté en *request* et *##er*, ce dernier étant marqué comme suffixe par la présence des dièses. De même, le terme *nf*, inconnu du vocabulaire, est décomposé en *n* et *##f*.

Les tokens obtenus sont ensuite convertis en identifiants (token IDs), qui sont fournis au modèle. Celui-ci produit en sortie des vecteurs encodés, représentant chaque token. Comme un mot peut être composé de plusieurs tokens, il est nécessaire d'agréger ces représentations. La méthode la plus simple consiste à effectuer la moyenne des encodages des tokens associés à un même mot. On obtient ainsi, au final, un embedding pour chaque mot.

1.5.4 Avantages et limites

Obtenir une proximité géométrique entre des éléments présentant une similarité sémantique constitue une propriété particulièrement intéressante dans notre contexte. En effet, de nombreux paramètres présentent une forte redondance lexicale ou fonctionnelle, comme *servingNfId* et *nf-id*. Ces paramètres peuvent ainsi être associés à des représentations proches dans l'espace latent, ce qui contribue à réduire la complexité du vocabulaire. De plus, l'utilisation de techniques de NLP pour enrichir la représentation sémantique des vecteurs d'arêtes peut fournir une information de départ supplémentaire au GNN, facilitant ainsi l'apprentissage de relations complexes.

Néanmoins, plusieurs limites apparaissent dans ce cadre. Pour réaliser l'embedding d'éléments textuels par NLP, il est possible soit d'utiliser directement des modèles pré-entraînés, soit de recourir à un fine-tuning afin de les adapter au vocabulaire spécifique du domaine. Dans notre cas, les paramètres à encoder s'apparentent davantage à des noms de variables qu'à des tokens issus du langage naturel. Par exemple, certains termes tels que *address* peuvent être correctement interprétés par les modèles, tandis que des identifiants techniques comme *avType* ou *kausf*, ne correspondent ni à des unités ni à des sous-unités lexicales courantes.

Dans ce contexte, le recours à une phase de fine-tuning apparaît nécessaire. Celle-ci consiste à fournir au modèle des exemples de phrases afin d'apprendre la sémantique des tokens à partir de leur contexte d'occurrence. Toutefois, dans notre cas, les données que l'on extrait des paquets ne sont pas naturellement structurées sous forme de phrases. La solution que nous considérons est donc de représenter chaque paquet comme une phrase en concaténant ses différents paramètres. Cette approche demeure cependant imparfaite, car elle introduit implicitement une notion d'ordre et de position dans la séquence, laquelle n'a pas de signification intrinsèque dans notre problématique.

Que ce soit pour FastText, GPT ou BERT, l'ordre des mots dans une phrase est généralement centrale dans les mécanismes d'apprentissage. Il est donc difficile de prédire dans quelle mesure ces modèles parviendront à capturer efficacement la sémantique de notre vocabulaire lorsque l'on considère un paquet non-ordonné comme une phrase. Nous évaluerons donc empiriquement, dans la section expérimentale présentée au chapitre suivant, les différents impacts des méthodes d'encodage afin d'identifier ceux qui permettent d'améliorer les performances de détection.

1.6 Conclusion

Nous avons vu dans ce chapitre que les besoins en détection d'anomalies dans les réseaux 5G s'éloignent fortement de ceux observés dans les réseaux traditionnels. Les spécificités de la 5G nécessitent de passer d'une analyse par flux à une analyse paquet, comme cela est réalisé dans la majorité des travaux portant sur la détection du domaine. Néanmoins, ces travaux prennent rarement en compte l'ensemble des

dimensions temporelles, sémantiques et topologiques que nous avons définies. En combinant les apports de l'état de l'art en 5G et des systèmes distribués, nous proposons un nouveau formalisme de graphe continu dynamique. Ce formalisme constitue l'une de nos principales contributions, car il permet de considérer ces trois dimensions tout en étant hautement interprétable par construction.

Nous avons ensuite présenté de manière détaillée le domaine relativement récent des GNN, ainsi que ses variantes basées sur les graphes dynamiques discrets et continus. Nous avons également montré que les deux types d'apprentissage présentent chacun leurs avantages. L'apprentissage non supervisé permet de détecter des attaques zero-day, mais nécessite l'utilisation de techniques issues du NLP pour gérer la forte dimensionnalité de ses paramètres. L'apprentissage supervisé, quant à lui, en plus d'obtenir généralement de meilleures performances, se concentre sur un sous-ensemble d'attaques connues et permet ainsi de réduire la dimensionnalité des paramètres à considérer, limitant le recours à des techniques de NLP.

Nous proposons dans le chapitre suivant une analyse empirique afin d'évaluer les performances des différents types d'apprentissage, des modèles de GNN ainsi que des différentes méthodes d'encodage discutées.

Bibliography

- [Ariu 2011] Davide Ariu, Roberto Tronci Giorgio Giacinto. *HMMPayL: An intrusion detection system based on Hidden Markov Models*. Computers & Security, vol. 30, no. 4, pages 221–241, June 2011. (Cité en page 4.)
- [Bandyopadhyay 2020] Sambaran Bandyopadhyay, Lokesh N, Saley Vishal Vivek M. N. Murty. *Outlier Resistant Unsupervised Deep Architectures for Attributed Network Embedding*. Proceedings of the 13th International Conference on Web Search and Data Mining, WSDM '20, pages 25–33, New York, NY, USA, January 2020. Association for Computing Machinery. (Cité en page 18.)
- [Bogatinovski 2021] Jasmin Bogatinovski Sasho Nedelkoski. *Multi-Source Anomaly Detection in Distributed IT Systems*, January 2021. (Cité en page 8.)
- [Bojanowski 2017] Piotr Bojanowski, Edouard Grave, Armand Joulin Tomas Mikolov. *Enriching Word Vectors with Subword Information*, June 2017. (Cité en pages 28 et 29.)
- [Brochier 2019] Robin Brochier, Adrien Guille Julien Velcin. *Global Vectors for Node Representations*. The World Wide Web Conference, pages 2587–2593, May 2019. (Cité en page 29.)
- [Chen 2021] Jun Chen Haopeng Chen. *Edge-Featured Graph Attention Network*, January 2021. (Cité en page 25.)
- [Cong 2023] Weilin Cong, Si Zhang, Jian Kang, Baichuan Yuan, Hao Wu, Xin Zhou, Hanghang Tong Mehrdad Mahdavi. *Do We Really Need Complicated Model Architectures For Temporal Networks?*, February 2023. (Cité en page 17.)
- [Devlin 2019] Jacob Devlin, Ming-Wei Chang, Kenton Lee Kristina Toutanova. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, May 2019. (Cité en page 29.)
- [Feng 2024] ZhengZhao Feng, Rui Wang, TianXing Wang, Mingli Song, Sai Wu Shuibing He. *A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges*, 2024. (Cité en page 17.)
- [Ganesh 2017] Meenu Ganesh, Priyanka Pednekar, Pooja Prabhuswamy, Divyashri Sreedharan Nair, Younghee Park Hyeran Jeon. *CNN-Based Android Malware Detection*, July 2017. (Cité en page 4.)
- [Gilmer 2017] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals George E. Dahl. *Neural Message Passing for Quantum Chemistry*, June 2017. (Cité en page 15.)

- [Hamilton 2018] William L. Hamilton, Rex Ying Jure Leskovec. *Inductive Representation Learning on Large Graphs*, 2018. (Cité en pages 8 et 15.)
- [Han 2022] Kai Han, Yunhe Wang, Jianyuan Guo, Yehui Tang Enhua Wu. *Vision GNN: An Image is Worth Graph of Nodes*. NeurIPS, 2022. (Cité en page 14.)
- [Hou 2022] Zhenyu Hou, Xiao Liu, Yukuo Cen, Yuxiao Dong, Hongxia Yang, Chunjie Wang Jie Tang. *GraphMAE: Self-Supervised Masked Graph Autoencoders*, July 2022. (Cité en page 18.)
- [Jiao 2024] Pengfe Jiao, Xinxun Zhang, Mengzhou Gao, Tianpeng Li Zhidong Zhao. *Informative Subgraphs Aware Masked Auto-Encoder in Dynamic Graphs*, sep 2024. (Cité en page 19.)
- [Kale 2023] Rahul Kale, Kar Wai Fok Vrizzlynn L. L. Thing. *Payload-based 5G Attack Detection*. 2023 9th International Conference on Computer and Communications (ICCC), pages 1262–1266, December 2023. (Cité en pages 4 et 7.)
- [Kipf 2017] Thomas N. Kipf Max Welling. *Semi-Supervised Classification with Graph Convolutional Networks*, 2017. (Cité en page 15.)
- [Koc 2012] Levent Koc, Thomas A. Mazzuchi Shahram Sarkani. *A network intrusion detection system based on a Hidden Naïve Bayes multiclass classifier*. Expert Systems with Applications, vol. 39, no. 18, pages 13492–13500, December 2012. (Cité en page 3.)
- [Kumar 2019] Srijan Kumar, Xikun Zhang Jure Leskovec. *Predicting Dynamic Embedding Trajectory in Temporal Interaction Networks*, August 2019. (Cité en pages 17 et 22.)
- [Lam 2020] Jordan Lam Robert Abbas. *Machine Learning based Anomaly Detection for 5G Networks*, March 2020. (Cité en pages 5 et 7.)
- [Liu 2010] Yan Liu, Ning Cao, Wei Pan Guangwei Qiao. *System anomaly detection in distributed systems through MapReduce-Based log analysis*. 2010 3rd International Conference on Advanced Computer Theory and Engineering(ICACTE), volume 6, pages V6–410–V6–413, August 2010. (Cité en page 8.)
- [Liu 2023] Yixin Liu, Shirui Pan, Yu Guang Wang, Fei Xiong, Liang Wang, Qingfeng Chen Vincent CS Lee. *Anomaly Detection in Dynamic Graphs via Transformer*. IEEE Transactions on Knowledge and Data Engineering, vol. 35, no. 12, pages 12081–12094, December 2023. (Cité en page 8.)
- [Madi 2021] Taous Madi, Hyame Assem Alameddine, Makan Pourzandi, Amine Boukhtouta, Moataz Shoukry Chadi Assi. *AutoGuard: A Dual Intelligence Proactive Anomaly Detection at Application-Layer in 5G Networks*. Elisa

- Bertino, Haya Shulman Michael Waidner, , Computer Security – ESORICS 2021, pages 715–735, Cham, 2021. Springer International Publishing. (Cité en pages 5 et 7.)
- [Mikolov 2013] Tomas Mikolov, Kai Chen, Greg Corrado Jeffrey Dean. *Efficient Estimation of Word Representations in Vector Space*, September 2013. (Cité en page 28.)
- [Moon 2017] Daesung Moon, Hyungjin Im, Ikkyun Kim Jong Park. *DTB-IDS: an intrusion detection system based on decision tree using behavior analysis for preventing APT attacks*. The Journal of Supercomputing, vol. 73, July 2017. (Cité en page 3.)
- [Münz 2007] Gerhard Münz, Sa Li G. Carle. *Traffic Anomaly Detection Using K-Means Clustering*. 2007. (Cité en page 4.)
- [Nguyen 2019] Thien Duc Nguyen, Samuel Marchal, Markus Miettinen, Hossein Fereidooni, N. Asokan Ahmad-Reza Sadeghi. *DIoT: A Federated Self-learning Anomaly Detection System for IoT*, May 2019. (Cité en page 8.)
- [Pacherkar 2024] Harsh Sanjay Pacherkar Guanhua Yan. *PROV5GC: Hardening 5G Core Network Security with Attack Detection and Attribution Based on Provenance Graphs*. Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WiSec '24, pages 254–264, New York, NY, USA, May 2024. Association for Computing Machinery. (Cité en pages 5, 6 et 7.)
- [Pareja 2019] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl Charles E. Leiserson. *EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs*, 2019. (Cité en page 16.)
- [Pazho 2024] Armin Danesh Pazho, Ghazal Alinezhad Noghre, Arnab A Purkayastha, Jagannadh Vempati, Otto Martin Hamed Tabkhi. *A Survey of Graph-Based Deep Learning for Anomaly Detection in Distributed Systems*. IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 1, pages 1–20, January 2024. (Cité en page 8.)
- [Protogerou 2021] Aikaterini Protogerou, Stavros Papadopoulos, Anastasios Drosou, Dimitrios Tzovaras Ioannis Refanidis. *A graph neural network method for distributed anomaly detection in IoT*. Evolving Systems, vol. 12, March 2021. (Cité en page 8.)
- [Radford 2018] Alec Radford, Karthik Narasimhan, Tim Salimans Ilya Sutskever. *Improving Language Understanding by Generative Pre-Training*. 2018. (Cité en page 29.)

- [Rossi 2020] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, Michael Bronstein. *Temporal Graph Networks for Deep Learning on Dynamic Graphs*, 2020. (Cité en pages 17, 19, 20, 21 et 22.)
- [Sankar 2019] Aravind Sankar, Yanhong Wu, Liang Gou, Wei Zhang, Hao Yang. *Dynamic Graph Representation Learning via Self-Attention Networks*, June 2019. (Cité en page 16.)
- [Scarselli 2009] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, Gabriele Monfardini. *The Graph Neural Network Model*. IEEE Transactions on Neural Networks, vol. 20, no. 1, pages 61–80, 2009. (Cité en page 15.)
- [Snort 2026] Snort. *Snort - Network Intrusion Detection & Prevention System*, 2026. (Cité en page 3.)
- [Suricata 2026] Suricata. *Home*, 2026. (Cité en page 3.)
- [Tan 2023] Yawen Tan, Jiadai Wang, Jiajia Liu, Yuanhao Li. *Deep Learning-Based Log Anomaly Detection for 5G Core Network*. 2023 IEEE/CIC International Conference on Communications in China (ICCC), pages 1–6, August 2023. (Cité en page 8.)
- [Tan 2025] Yawen Tan, Jiajia Liu, Yuanhao Li, Jiadai Wang. *Deep Learning Based Proactive Anomaly Detection for 5G Core Control Plane Network Function Interactions*. IEEE Transactions on Cognitive Communications and Networking, pages 1–1, 2025. (Cité en pages 5 et 7.)
- [Tian 2023] Zixu Tian, Rajendra Patil, Mohan Gurusamy, Joshua McCloud. *ADSeq-5GCN: Anomaly Detection from Network Traffic Sequences in 5G Core Network Control Plane*. 2023 IEEE 24th International Conference on High Performance Switching and Routing (HPSR), pages 75–82, June 2023. (Cité en pages 5 et 7.)
- [Trivedi 2018] Rakshit Trivedi, Mehrdad Farajtabar, Prasenjeet Biswal, Hongyuan Zha. *Representation Learning over Dynamic Graphs*. International Conference on Learning Representations (ICLR), 2018. (Cité en page 17.)
- [Veličković 2018] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, Yoshua Bengio. *Graph Attention Networks*, 2018. (Cité en pages 15 et 16.)
- [Vijayanand 2018] R. Vijayanand, D. Devaraj, B. Kannapiran. *Intrusion detection system for wireless mesh network using multiple support vector machine classifiers with genetic-algorithm-based feature selection*. Computers & Security, vol. 77, pages 304–314, August 2018. (Cité en page 3.)

- [Wang 2022] Yanbang Wang, Yen-Yu Chang, Yunyu Liu, Jure Leskovec Pan Li. *Inductive Representation Learning in Temporal Networks via Causal Anonymous Walks*, October 2022. (Cité en pages 16 et 17.)
- [Wang 2023] Min Wang, Peng Li, Zhang Cheng, Wenmao Liu, Lei Nie, Haizhou Bao, Qin Liu Kai Zhang. *Unsupervised Graph-Sequence Anomaly Detection for 5G Core Network Control Plane Traffic*. 2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS), pages 1645–1652, December 2023. (Cité en pages 6, 7 et 19.)
- [Wei 2024] Kang Wei, Jun Li, Chuan Ma, Ming Ding, Sha Wei, Fan Wu, Guihai Chen Thilina Ranbaduge. *Vertical Federated Learning: Challenges, Methodologies and Experiments*, August 2024. (Cité en page 8.)
- [Xu 2019] Keyulu Xu, Weihua Hu, Jure Leskovec Stefanie Jegelka. *How Powerful are Graph Neural Networks?*, 2019. (Cité en page 15.)
- [Xu 2020] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar Kannan Achan. *INDUCTIVE REPRESENTATION LEARNING ON TEMPORAL GRAPHS*. International Conference on Learning Representations, 2020. (Cité en pages 16 et 17.)
- [Yu 2023] Le Yu, Leilei Sun, Bowen Du Weifeng Lv. *Towards Better Dynamic Graph Learning: New Architecture and Unified Library*, March 2023. (Cité en page 17.)
- [Zeek 2026] Zeek. *The Zeek Network Security Monitor*, 2026. (Cité en page 3.)
- [Zhang 2019] Wang Zhang Yin Yiu. *Graph Masked Autoencoder for Spatio-Temporal Graph Learning*, 2019. (Cité en page 19.)
- [Zhao 2020] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng Haifeng Li. *T-GCN: A Temporal Graph ConvolutionalNetwork for Traffic Prediction*. IEEE Transactions on Intelligent Transportation Systems, vol. 21, no. 9, pages 3848–3858, 2020. (Cité en page 16.)